



ROYAUME DU MAROC

المملكة المغربية

,Ministère de l'Enseignement Supérieur,
de la Formation des Cadres et de la Recherche Scientifique

CONCOURS NATIONAL COMMUN

d'Admission dans les Établissements de Formation

d'Ingénieurs et Établissements Assimilés

Édition 2021 - 2014

ÉPREUVE D'INFORMATIQUE

Partie : Calcul Numérique

Préparation CNC : Informatique

Filières : **MP/PSI/TSI**

Durée 2 heures

Partie I : Calcul numérique

Modélisation de la propagation d'un virus

Le modèle 'SEIR'

La crise sanitaire mondiale du Coronavirus Covid-19 a démontré le rôle des modélisations mathématiques dans la prise de décisions politiques et sanitaires.

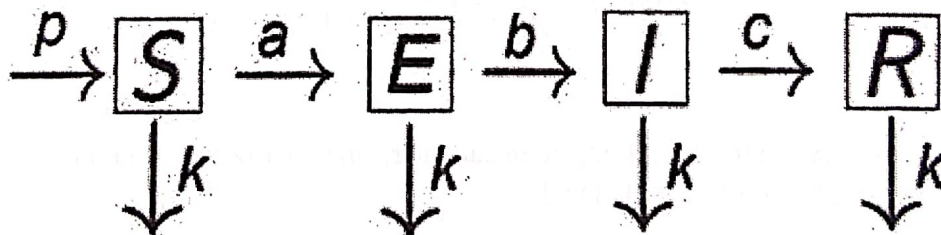
Les modèles à compartiments font partie des premiers modèles mathématiques, à avoir été utilisés en épidémiologie. L'idée est de diviser une population en plusieurs compartiments (groupes d'individus), et définir les règles d'échanges entre ces compartiments.

Le modèle SEIR

Dans ce modèle, les individus sont répartis en 4 compartiments :

- ❖ S, dans lequel se trouvent les individus sains et susceptibles d'être infectés ;
- ❖ E, pour les individus infectés qui ne sont pas contagieux ;
- ❖ I, pour les individus infectés qui sont contagieux ;
- ❖ R, pour les individus retirés du modèle, non susceptibles d'être infectés, car guéris et immunisés.

Le modèle **SEIR** décrit l'évolution dans le temps du nombre d'individus dans chaque compartiment, et part du schéma suivant :



- p : représente le taux de natalité ;
- a : représente le taux de transmission de la maladie d'un individu infecté à un individu sain ;
- b : représente le taux d'incubation de la maladie ;
- c : représente le taux de guérison ;
- k : représente le taux de mortalité.

Il paraît plus naturel de travailler avec le nombre de personnes dans chaque catégorie, mais certains calculs seront plus simples si on utilise plutôt la proportion de personnes dans chaque catégorie, ce qui nous permet de connaître tout aussi bien la progression de l'épidémie.

On note donc : $S(t)$, $E(t)$, $I(t)$ et $R(t)$ les *proportions* des individus dans chaque catégories à l'instant t .

L'évolution des 4 catégories de population peut alors être décrite par le système d'équations différentielles suivant :

$$(1) \quad \begin{cases} S'(t) = -a.S(t).I(t) + p.N(t) - k.S(t) \\ E'(t) = a.S(t).I(t) - (b+k).E(t) \\ I'(t) = b.E(t) - (c+k).I(t) \\ R'(t) = c.I(t) - k.R(t) \end{cases} \quad \text{avec : } N(t) = S(t) + E(t) + I(t) + R(t)$$

Les dérivées permettent de connaître la variation (c'est à dire si c'est croissant ou décroissant) des fonctions $S(t)$, $E(t)$, $I(t)$ et $R(t)$ en fonction du temps t , afin d'en décrire l'évolution au cours du temps.

On suppose que les modules **numpy** et **matplotlib.pyplot** sont importés :

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

Méthodes de Runge-Kutta :

Dans cette partie, nous allons nous intéresser à une méthode d'ordre 4, appelée **Runge-Kutta 4 (RK4)**, permettant de résoudre numériquement les équations différentielles du premier ordre, avec une condition initiale, sous la forme :

$$\begin{cases} y' = f(t, y(t)) & ; \quad \forall t \in [t_0, t_0 + T] \\ y(t_0) = y_0 \end{cases}$$

On note $I = [t_0, t_0 + T]$ l'intervalle de résolution. Pour un nombre de nœuds N donné, soit $t_n = t_0 + n \cdot h$, avec $n = 0, 1, 2, \dots, N$, une suite de nœuds de I induisant une discrétisation de I en sous-intervalles $I_n = [t_n, t_{n+1}]$. La longueur h de ces sous-intervalles est appelée pas de discrétisation. Le pas de discrétisation h est donné par : $h = \frac{T}{N}$.

Soit y_n l'approximation de la solution exacte $y(t_n)$ au nœud t_n . Le schéma de la méthode **RK4** est donné par :

- y_0 donné
- $y_{n+1} = y_n + \frac{h}{6} * (k_1 + 2(k_2 + k_3) + k_4)$

Avec :

- $k_1 = f(t_n, y_n)$
- $k_2 = f(t_n + \frac{h}{2}, y_n + k_1 * \frac{h}{2})$
- $k_3 = f(t_n + \frac{h}{2}, y_n + k_2 * \frac{h}{2})$
- $k_4 = f(t_n + h, y_n + k_3 * h)$

Q.1- Écrire la fonction `def RK4(f, t0, T, y0, N)`: qui reçoit en paramètres la fonction $f: (t, y) \rightarrow f(t, y)$, t_0 et T tels que $[t_0, t_0 + T]$ est l'intervalle de résolution, la condition initiale y_0 et N est le nombre de nœuds. La fonction retourne $tt = [t_0, t_1, \dots, t_N]$ et $Y = [y_0, y_1, \dots, y_N]$ en utilisant le schéma de RK4.

Q.2- Le but de cette question est d'utiliser la fonction précédente **RK4**, pour résoudre numériquement le système d'équations différentielles (1).

Q.2.a- En posant $Y(t) = [S(t), E(t), I(t), R(t)]$, montrer que le système d'équations différentielles (1) peut s'écrire sous la forme $Y'(t) = F(t, Y(t))$ avec :

$$\begin{aligned} F &: \mathbb{I} * \mathbb{R}^4 \rightarrow \mathbb{R}^4 \\ (t, X = [X[0], X[1], X[2], X[3]]) &\rightarrow [z_0, z_1, z_2, z_3] \end{aligned}$$

z_0, z_1, z_2 et z_3 sont à préciser en fonction de $X[0], X[1], X[2]$ et $X[3]$.

Q.2.b- On suppose que p, a, b, c et k sont des variables globales initialisées par les valeurs suivantes :

$p = 0.004$; $a = 0.8$; $b = 0.5$; $c = 0.09$; $k = 0.005$

Écrire, en langage Python, la fonction `def F(t, X)`: qui reçoit en paramètres un réel t , et un tableau `np.array X` de dimension 1 ligne et 4 colonnes. La fonction retourne le tableau `np.array` de même dimension, image de (t, X) par la fonction F .

Q.3- En utilisant la fonction précédente **RK4**, écrire un script python qui permet de tracer la représentation graphique ci-dessous (Figure 1), des fonctions $S(t)$, $E(t)$, $I(t)$, $R(t)$ et $N(t)$. Le nombre de points générés dans chaque courbe est $N=10^3$.

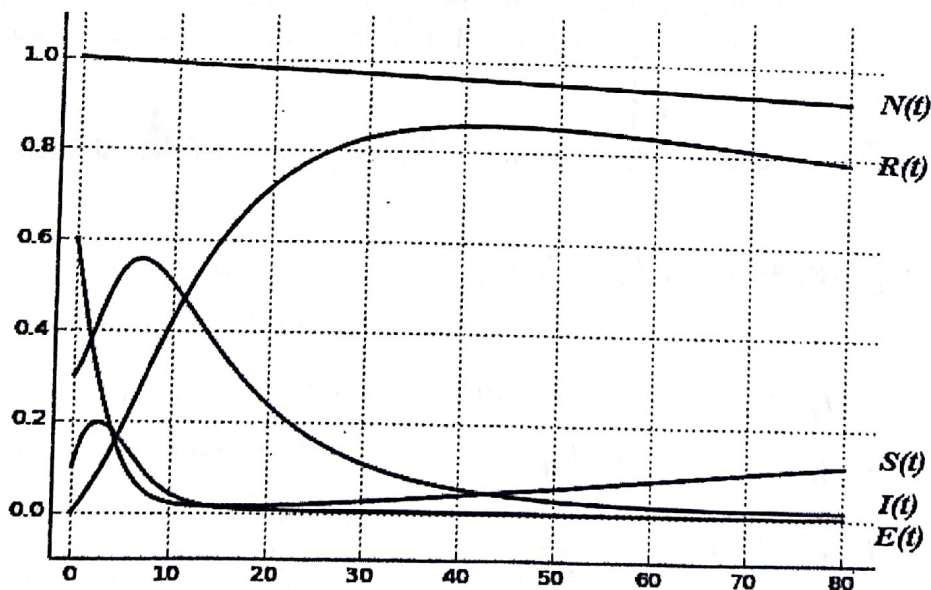


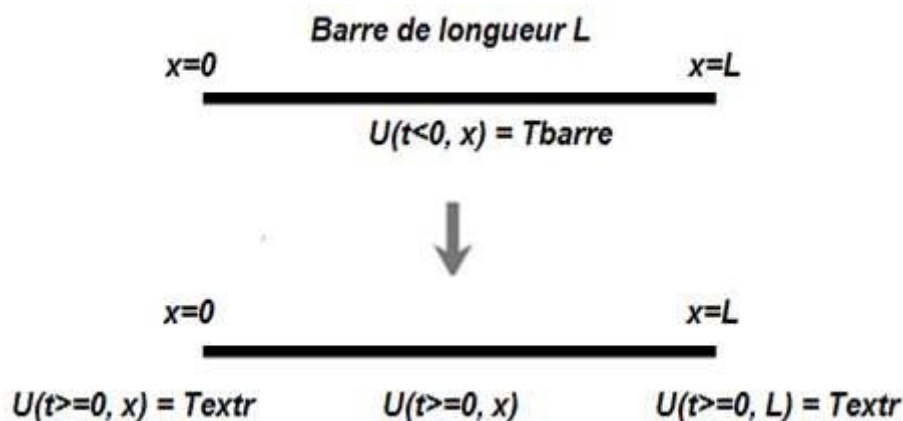
Figure 1

Partie I : Calcul numérique

Équation de la diffusion thermique

On considère une barre solide de longueur L , de coefficient de diffusion thermique D .

La barre est initialement "préparée" dans un état de température T_{barre} . Les deux extrémités de la barre sont maintenues à une température extérieure constante T_{extr} .



L'équation de la diffusion thermique à une dimension, est la suivante :

$$\frac{\partial U}{\partial t} = D \frac{\partial^2 U}{\partial x^2}$$

Avec : $U(x, t)$ est la température de la position x dans la barre, à un instant t .

L'équation (E) admet une solution unique :

$$U(x, t) = \sum_{n=1}^{\infty} \left[\frac{2(T_{barre} - T_{extr})}{n\pi} \sin\left(\frac{n\pi x}{L}\right) \exp\left(-\frac{n^2\pi^2 D t}{L^2}\right) + T_{extr} \right]$$

On suppose que les variables globales suivantes, sont déclarées et initialisées par les valeurs suivantes :

$L = 1.0$	Longueur de la barre
$D = 0.2$	Coefficient de diffusion thermique
$T = 1.4$	Durée totale de l'évolution de la température
$T_{barre} = 100.0$	Température de la barre
$T_{extr} = 20.0$	Température extérieure

Dans cette partie, on suppose que les modules **numpy** et **matplotlib.pyplot** sont importés :

```
import numpy np
import matplotlib.pyplot as plt
```

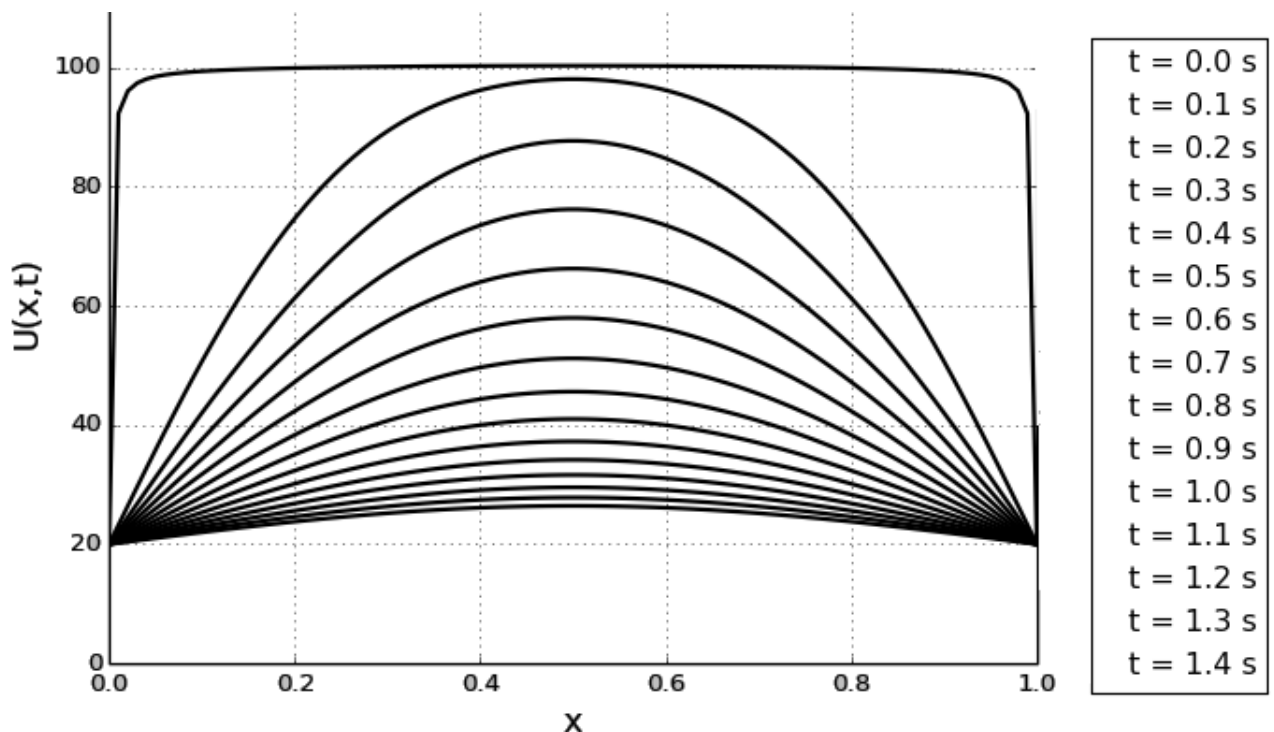
Q.1- Écrire, en Python, la fonction f définie par :

$$f(x, y) = -\left(1 - \frac{1}{x}\right) \cdot \left(\frac{1}{y}\right) \cdot \left(\frac{1}{x}\right) \cdot \left(\frac{1}{y}\right)$$

Q.2- Écrire la fonction $U(x, t)$ qui reçoit en paramètres deux réels x et t . La fonction retourne la valeur de $U(x, t)$ qui correspond à la somme partielle d'indice $m=200$:

$$U(x, t) = \sum_{m=0}^{200} \left(\frac{1}{m+1} \cdot \left(\cos\left(\frac{(m+1)\pi x}{2}\right) \cdot \exp\left(-\frac{(m+1)^2 \pi^2 t}{4}\right) \right) \right)$$

Q.3- Écrire le programme permettant de tracer la représentation graphique suivante, qui représente l'évolution de la température dans les points x de la barre, à intervalle de temps égal à $0.1s$, sachant que la barre est subdivisée en **100** points, et l'indice de la somme partielle est $m=200$.



NB : Chaque courbe représente la température à chaque point x de la barre à un instant t .

Partie I : Calcul numérique

Intégration numérique

Méthode de "Simpson"

En analyse numérique, il existe une vaste famille d'algorithmes dont le but principal est d'estimer la valeur numérique de l'intégrale d'une fonction f sur un intervalle $[a, b]$:

$$\int_a^b f(x) dx$$

Ces techniques procèdent en trois phases :

- Décomposition de l'intervalle $[a, b]$ en sous-intervalles contigus ;
- Intégration approchée de la fonction sur chaque sous-intervalle ;
- Sommation des résultats numériques ainsi obtenus.

Méthode de Simpson :

La **méthode de Simpson**, du nom de Thomas Simpson, est une technique qui utilise l'approximation d'ordre 2 de f par un polynôme quadratique P prenant les mêmes valeurs que f aux points d'abscisses a , $m = \frac{a+b}{2}$ et b .

Pour déterminer l'expression de cette parabole (polynôme de degré 2), on utilise l'interpolation lagrangienne. Le résultat peut être mis sous la forme suivante :

$$P(x) = f(a) \frac{(x-m)(x-b)}{(a-m)(a-b)} + f(m) \frac{(x-a)(x-b)}{(m-a)(m-b)} + f(b) \frac{(x-a)(x-m)}{(b-a)(b-m)}$$

Un polynôme étant une fonction très facile à intégrer, on approche l'intégrale de la fonction f sur l'intervalle $[a, b]$, par l'intégrale de P sur ce même intervalle. On a ainsi, la formule simple :

$$\int_a^b f(x) dx \approx \int_a^b P(x) dx = \frac{b-a}{6} (f(a) + 4f\left(\frac{a+b}{2}\right) + f(b))$$

Pour appliquer cette méthode d'intégration, on doit découper l'intervalle $[a, b]$ en n sous-intervalles de longueur $h = \frac{b-a}{n}$. Ensuite on applique la formule précédente sur chacun des sous-intervalles, et on effectue la somme des résultats obtenus. En simplifiant la sommation des résultats obtenus, on obtient la formule finale suivante :

$$\int_a^b f(x) dx \approx \frac{h}{6} (f(a) + 4f\left(a + \frac{h}{2}\right) + f(b)) + \frac{h}{3} \sum_{i=1}^{n-1} f(a + ih) + 2f\left(a + ih + \frac{h}{2}\right)$$

Q.1- Écrire la fonction `somme(f, a, h, n)` qui retourne la valeur de la somme suivante.

$$\sum_{i=1}^{n-1} f(a + ih) + 2f\left(a + ih + \frac{h}{2}\right)$$

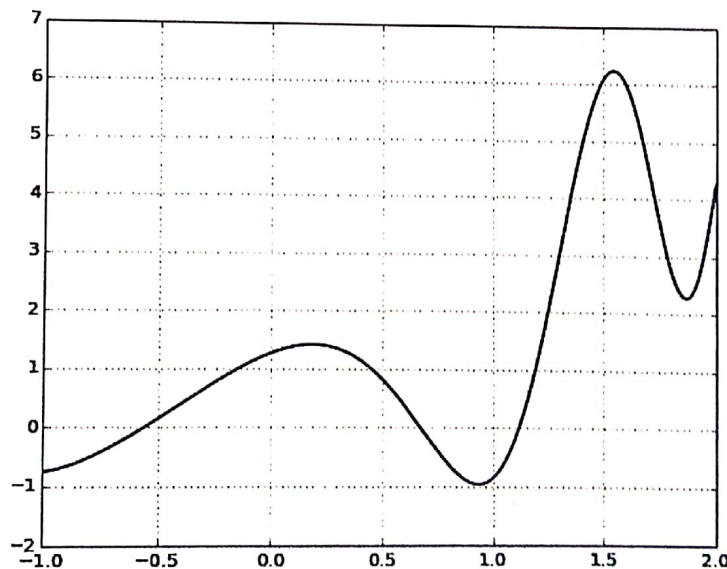
Q.2- Écrire la fonction `integrale_simpson(f, a, b, n)`, qui retourne la valeur de l'intégrale de la fonction f dans l'intervalle $[a, b]$, en utilisant la formule de simpson.

On suppose que les modules `numpy` et `matplotlib.pyplot` sont importés :

```
import numpy as np
import matplotlib.pyplot as plt
```

Q.3.a – Écrire en python, la fonction g définie par : $g(x) = x^2 + 5 \cdot \cos(1+x^2) \cdot \sin(e^x) - 1$

Q.3.b – Écrire le programme qui trace la courbe suivante de la fonction g . Le nombre de points générés dans la courbe est : 250



Q.3.c- Écrire le code python qui utilise la méthode de simpson, et qui calcule et affiche la valeur de l'intégral de la fonction g , dans l'intervalle $[-0.5, 1.5]$, en utilisant pour nombre de subdivision : $n=10^5$

Partie I : Calcul numérique

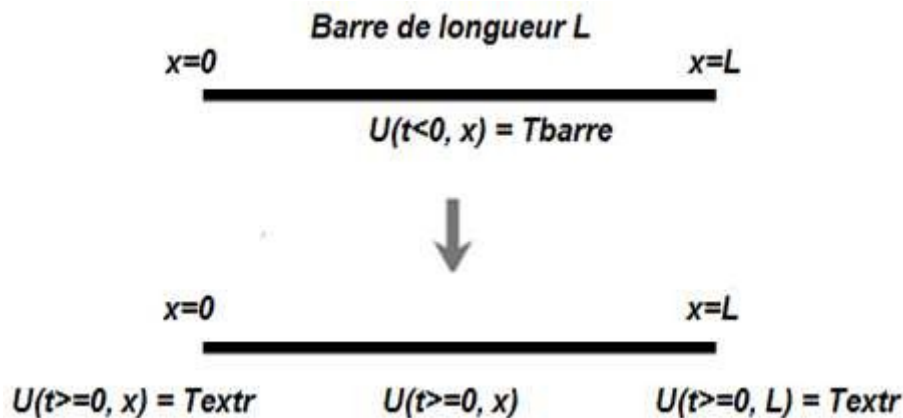
Équation de la diffusion thermique

Solution numérique

On considère une barre solide de longueur L , de coefficient de diffusion thermique D .

La barre est initialement "préparée" dans un état de température T_{barre} .

Les deux extrémités de la barre sont maintenues à une température extérieure constante T_{extr} .



L'équation de la diffusion thermique à une dimension, est la suivante :

$$\frac{\partial U}{\partial t} = D \frac{\partial^2 U}{\partial x^2}$$

Avec : $U(t, x)$ est la température de la position x dans la barre de la barre, à l'instant t .

On recherche une solution numérique à ce problème par la méthode classique **des différences finies**.

Supposons que nous cherchions l'évolution de $U(t, x)$ sur une durée totale T .

La durée totale T d'évolution est subdivisée en n sous-intervalles de durée : $\Delta t = \frac{T}{n}$

Ainsi, l'instant "discret" t_i est : $t_i = i * \Delta t$ avec $i \in \{0, 1, \dots, n\}$,

De même, la barre de longueur L est spatialement subdivisée en p tronçons de longueur : $\Delta x = \frac{L}{p}$

Ainsi, l'abscisse "discrète" x_i est : $x_i = i * \Delta x$ avec $i \in \{0, 1, \dots, p\}$,

On suppose que les variables globales suivantes, sont déclarées et initialisées :

$L = 1.0$	La longueur de la barre
$D = 0.3$	Le coefficient de diffusion thermique
$T = 1.0$	La durée totale de l'évolution
$T_{barre} = 100.0$	La température de la barre
$T_{extr} = 20.0$	La température extérieure
$n = 10000$	Le nombre de subdivision de T
$p = 100$	Le nombre de subdivision de L

On suppose que les modules **numpy** et **matplotlib.pyplot** sont importés :

```
import numpy as np
import matplotlib.pyplot as plt
```

Q.1- Écrire la fonction **vecteur_init ()** qui retourne un vecteur U de longueur $p+1$, tel que :

$$U[i] = T_{\text{extr}} \quad \text{si } i = 0 \text{ ou } i = p$$

$$U[i] = T_{\text{barre}} \quad \text{si } 1 \leq i \leq p-1$$

La fonction **vecteur_init ()** retourne le vecteur U suivant :

[20. 100. 100. 100. ... 100. 100. 100. 20.]

Chaque élément $U[i]$ représente la température du point $i \cdot dx$ dans la barre, à l'instant $t_0 = 0 \text{ s}$.

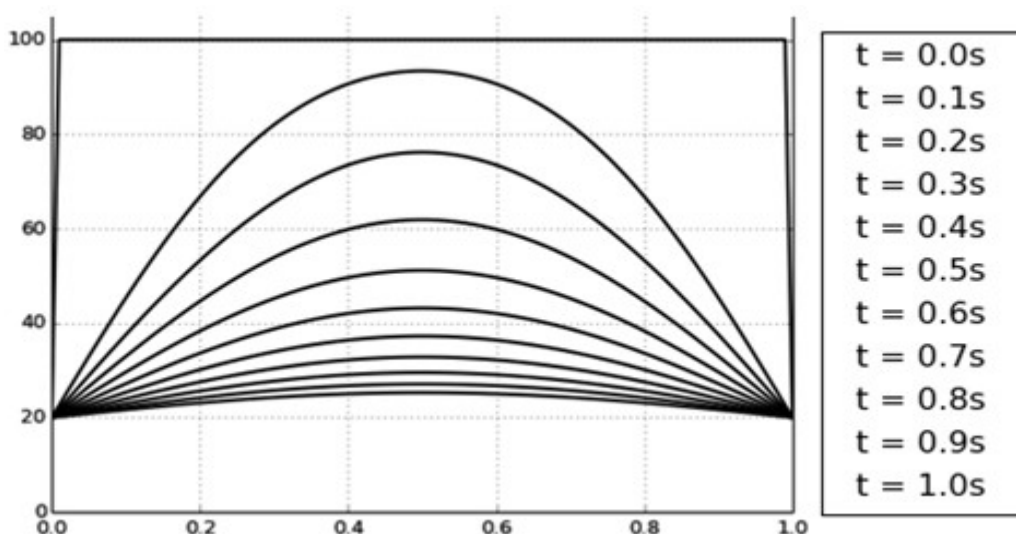
L'équation de la diffusion thermique (E) discrétisée s'écrit sous forme d'une relation de récurrence qui permet d'obtenir la température de la barre à l'instant t_{i+1} en fonction de la température de la barre à l'instant t_i : Si le vecteur U représente la température de la barre à l'instant t_i , alors la température de la barre à l'instant t_{i+1} est représentée par le vecteur V , tel que :

$$\text{Si } i = 0 \text{ ou } i = p \text{ alors } [] \quad \neq []$$

$$\text{Si } 1 \leq i \leq p-1 \text{ alors } [] \quad \neq [] + * \text{---} * [- \#] - * [] + [+ \#]$$

Q.2- Écrire la fonction **differences_finies(U)** qui reçoit en paramètre un vecteur U qui représente la température de la barre à un instant t_i . La fonction retourne le vecteur V qui représente la température de la barre à l'instant t_{i+1} .

Q.3- Écrire le programme qui trace la représentation graphique de l'évolution de la température de la barre, à intervalle de temps égal à 0.1 s



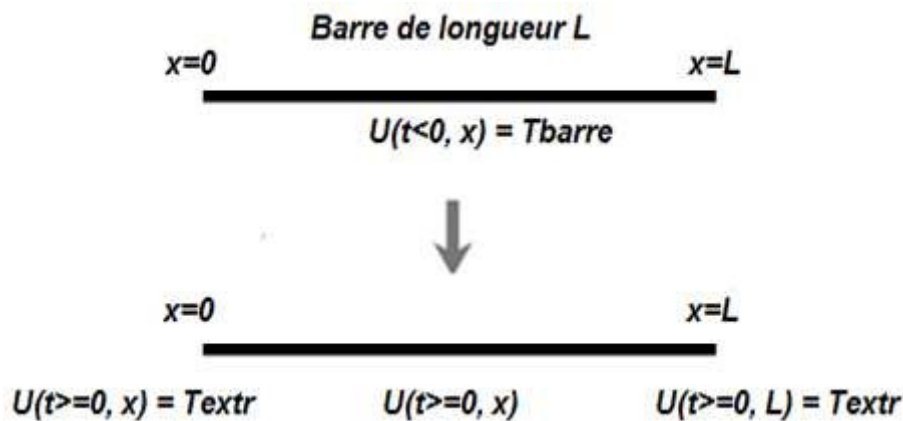
NB : Chaque courbe représente la température de la barre à un instant t

Partie I : Calcul numérique

Équation de la diffusion thermique

On considère une barre solide de longueur L , de coefficient de diffusion thermique D .

La barre est initialement "préparée" dans un état de température T_{barre} . Les deux extrémités de la barre sont maintenues à une température extérieure constante T_{extr} .



L'équation de la diffusion thermique à une dimension, est la suivante :

$$\frac{\partial U(x, t)}{\partial t} = D \frac{\partial^2 U(x, t)}{\partial x^2}$$

Avec : $U(x, t)$ est la température de la position x dans la barre, à un instant t .

L'équation (E) admet une solution unique :

$$U(x, t) = T_{extr} + \sum_{n=1}^{\infty} \left[\frac{T_{barre} - T_{extr}}{n\pi} \sin\left(\frac{n\pi x}{L}\right) e^{-\frac{n^2\pi^2 D t}{L^2}} \right]$$

On suppose que les variables globales suivantes, sont déclarées et initialisées par les valeurs suivantes :

$L = 1.0$	Longueur de la barre
$D = 0.2$	Coefficient de diffusion thermique
$T = 1.4$	Durée totale de l'évolution de la température
$T_{barre} = 100.0$	Température de la barre
$T_{extr} = 20.0$	Température extérieure

Dans cette partie, on suppose que les modules **numpy** et **matplotlib.pyplot** sont importés :

```
import numpy np
import matplotlib.pyplot as plt
```

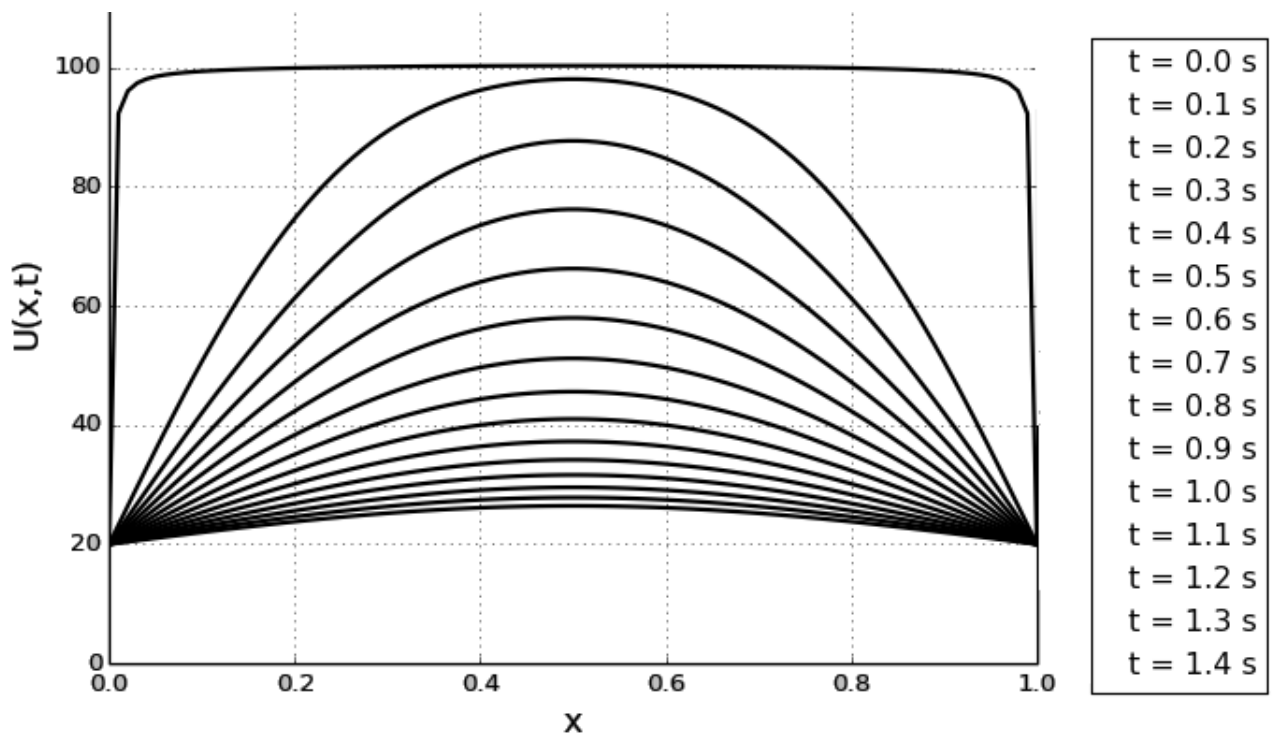
Q.1- Écrire, en Python, la fonction f définie par :

$$f(x, y) = -\left(1 - \frac{1}{x}\right) \cdot \left(\frac{1}{y}\right) \cdot \left(\frac{1}{x}\right) \cdot \left(\frac{1}{y}\right)$$

Q.2- Écrire la fonction $U(x, t)$ qui reçoit en paramètres deux réels x et t . La fonction retourne la valeur de $U(x, t)$ qui correspond à la somme partielle d'indice $m=200$:

$$U(x, t) = \sum_{m=0}^{200} \left(\frac{1}{m} \cdot \left(\frac{1}{x} + \frac{1}{y} \right) \right) \cdot f(x, y)$$

Q.3- Écrire le programme permettant de tracer la représentation graphique suivante, qui représente l'évolution de la température dans les points x de la barre, à intervalle de temps égal à $0.1s$, sachant que la barre est subdivisée en **100** points, et l'indice de la somme partielle est $m=200$.



NB : Chaque courbe représente la température à chaque point x de la barre à un instant t .

Partie I : Calcul numérique

Intégration numérique

Méthode de "Simpson"

En analyse numérique, il existe une vaste famille d'algorithmes dont le but principal est d'estimer la valeur numérique de l'intégrale d'une fonction f sur un intervalle $[a, b]$:

Ces techniques procèdent en trois phases :

- Décomposition de l'intervalle $[a, b]$ en sous-intervalles contigus ;
- Intégration approchée de la fonction sur chaque sous-intervalle ;
- Sommation des résultats numériques ainsi obtenus.

Méthode de Simpson :

La **méthode de Simpson**, du nom de Thomas Simpson, est une technique qui utilise l'approximation d'ordre 2 de f par un polynôme quadratique P prenant les mêmes valeurs que f aux points d'abscisses a , $x = \frac{a+b}{2}$ et b .

Pour déterminer l'expression de cette parabole (polynôme de degré 2), on utilise l'interpolation lagrangienne. Le résultat peut être mis sous la forme suivante :

$$= \frac{f(a) \cdot \frac{b-x}{2}}{\frac{b-a}{2}} + \frac{f(x) \cdot \frac{(x-a)(x-b)}{2}}{\frac{(b-a)^2}{4}} + \frac{f(b) \cdot \frac{x-a}{2}}{\frac{b-a}{2}}$$

Un polynôme étant une fonction très facile à intégrer, on approche l'intégrale de la fonction f sur l'intervalle $[a, b]$, par l'intégrale de P sur ce même intervalle. On a ainsi, la formule simple :

$$\approx \frac{b-a}{6} \left(f(a) + 4f\left(\frac{a+b}{2}\right) + f(b) \right)$$

Pour appliquer cette méthode d'intégration, on doit découper l'intervalle $[a, b]$ en n sous-intervalles de longueur $\frac{b-a}{n}$. Ensuite on applique la formule précédente sur chacun des sous-intervalles, et on effectue la somme des résultats obtenus. En simplifiant la sommation des résultats obtenus, on obtient la formule finale suivante :

$$\frac{b-a}{n} \left(\frac{1}{3} f(a) + \frac{4}{3} \sum_{i=1}^{n-1} f\left(a + (i-\frac{1}{2}) \frac{b-a}{n}\right) + \frac{1}{3} f(b) \right)$$

Q.1- Écrire la fonction `somme(f, a, h, n)` qui retourne la valeur de la somme suivante.

$$+ \quad + \quad + \quad + -$$

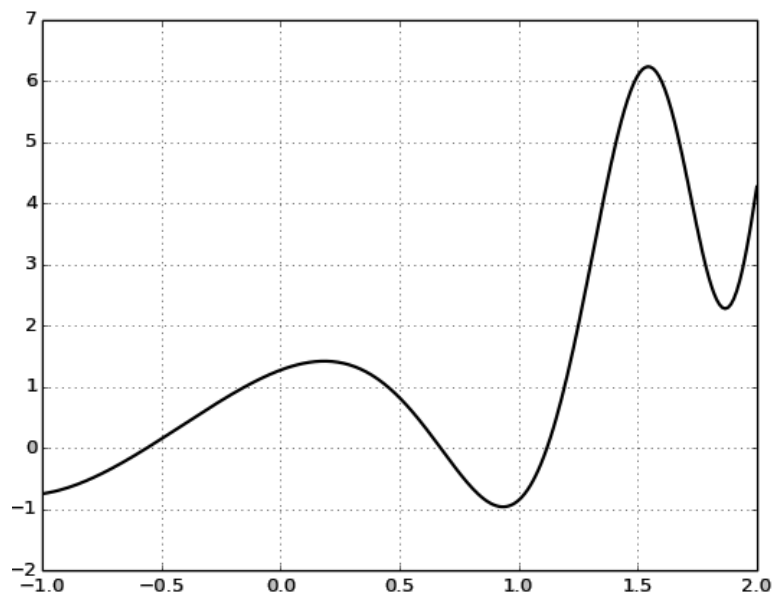
Q.2- Écrire la fonction `integrale_simpson(f, a, b, n)`, qui retourne la valeur de l'intégrale de la fonction f dans l'intervalle $[a, b]$, en utilisant la formule de simpson.

On suppose que les modules `numpy` et `matplotlib.pyplot` sont importés :

```
import numpy as np
import matplotlib.pyplot as plt
```

Q.3.a – Écrire en python, la fonction g définie par : $g(x) = x^2 + 5*\cos(1+x^2)*\sin(e^x) - 1$

Q.3.b – Écrire le programme qui trace la courbe suivante de la fonction g . Le nombre de points générés dans la courbe est : **250**



Q.3.c- Écrire le code python qui utilise la méthode de simpson, et qui calcule et affiche la valeur de l'intégral de la fonction g , dans l'intervalle $[-0.5, 1.5]$, en utilisant pour nombre de subdivision : $n=10^5$

Partie II :

Grille binaire

Une **grille binaire** de **n** lignes et **p** colonnes, est une grille dans laquelle chaque case est de **couleur blanche**, ou bien de **couleur noire**.

Exemple :

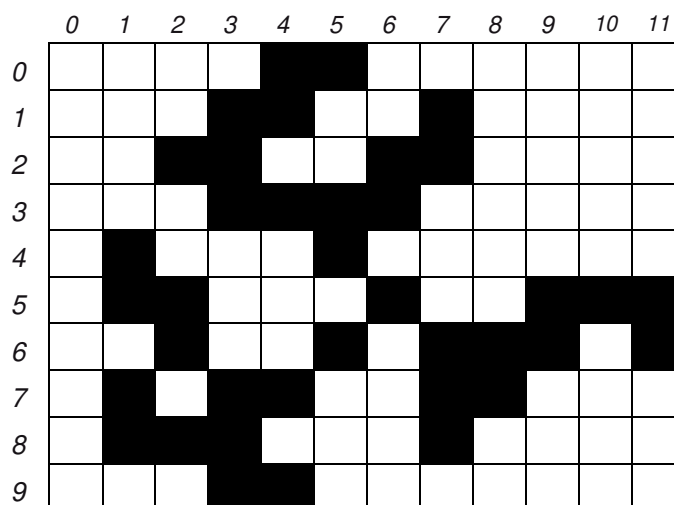


Figure 1 : Exemple de grille binaire de **10** lignes et **12** colonnes

Représentation de la grille binaire :

Pour représenter une grille binaire de **n** lignes et **p** colonnes, on utilise une matrice **G** de **n** lignes et **p** colonnes.

Chaque case de la grille **G** est représentée par un tuple **(i, j)** avec $0 \leq i < n$ et $0 \leq j < p$, tels que :

$G[i, j] = 1$, si la couleur de la case **(i, j)** est **blanche** ;

$G[i, j] = 0$, si la couleur de la case **(i, j)** est **noire**.

Exemple :

La grille binaire de la **figure 1** est représentée par la matrice **G** de **10** lignes et **12** colonnes, suivante :

1	1	1	1	0	0	1	1	1	1	1	1
1	1	1	0	0	1	1	0	1	1	1	1
1	1	0	0	1	1	0	0	1	1	1	1
1	1	1	0	0	0	0	1	1	1	1	1
1	0	1	1	1	0	1	1	1	1	1	1
1	0	0	1	1	1	0	1	1	0	0	0
1	1	0	1	1	0	1	0	0	0	1	0
1	0	1	0	0	1	1	0	0	1	1	1
1	0	0	0	1	1	1	0	1	1	1	1
1	1	1	0	0	1	1	1	1	1	1	1

II. 1- Calcul du déterminant d'une grille binaire carrée

Une grille binaire **carrée** est une grille binaire dans laquelle le nombre de lignes et le nombre de colonnes sont égaux.

Exemple : Grille binaire carrée d'ordre **10** (10 lignes et 10 colonnes).

	0	1	2	3	4	5	6	7	8	9
0										
1										
2										
3										
4										
5										
6										
7										
8										
9										

Dans cette section (II. 1), on suppose que le module **numpy** est importé :

from numpy import *

On suppose que la matrice carrée **G**, qui représente la grille binaire carrée, est créée par la méthode **array()** du module **numpy**.

Exemple :

La grille binaire carrée d'ordre **10** est représentée par la matrice carrée **G** suivante :

```
G = array ([ [1, 1, 1, 1, 0, 0, 1, 1, 1, 1] ,
             [1, 1, 1, 0, 0, 1, 1, 0, 1, 1] ,
             [1, 1, 0, 0, 1, 1, 0, 0, 1, 1] ,
             [1, 1, 1, 0, 0, 0, 0, 1, 1, 1] ,
             [1, 0, 1, 1, 1, 0, 1, 1, 1, 1] ,
             [1, 0, 0, 1, 1, 1, 0, 1, 1, 0] ,
             [1, 1, 0, 1, 1, 0, 1, 0, 0, 0] ,
             [1, 0, 1, 0, 0, 1, 1, 0, 0, 1] ,
             [1, 0, 0, 0, 1, 1, 1, 0, 1, 1] ,
             [1, 1, 1, 0, 0, 1, 1, 1, 1, 1] ] , float)
```

Dans le but de calculer le déterminant d'une matrice carrée **G** qui représente une grille binaire carrée, on propose d'utiliser la méthode du '*pivot de Gauss*', dont le principe est le suivant :

1. Créer une matrice **C** copie de la matrice **G** ;
2. En utilisant la méthode du *pivot de Gauss*, transformer la matrice **C** en matrice triangulaire inférieure, ou bien en matrice triangulaire supérieure, en comptant le nombre d'échanges de lignes dans la matrice **C**. On pose **k** le nombre d'échanges de lignes dans la matrice **C** ;
3. Calculer le déterminant **D** de la matrice triangulaire **C** ;
4. Le déterminant de la matrice **M** est égal à : $\ast (-1)^k$.

Q 1. a- Écrire la fonction **copie_matrice(G)**, qui reçoit en paramètre une matrice carrée **G** qui représente une grille binaire carrée, et qui renvoie une matrice **C** copie de la matrice **G**.

Q 1. b- Écrire la fonction **echange_lignes(C, i, j)**, qui reçoit en paramètres une matrice carrée **C** qui représente une grille binaire carrée. La fonction échange les lignes **i** et **j** dans la matrice **C**.

Q 1. c- Écrire la fonction **triangulaire(C)**, qui reçoit en paramètre une matrice carrée **C** qui représente une grille binaire carrée. En utilisant la méthode du Pivot de Gauss, la fonction transforme la matrice **C** en matrice triangulaire inférieure ou bien triangulaire supérieure, tout en comptant le nombre de fois qu'il y a eu échange de lignes dans la matrice **C**. La fonction doit retourner le nombre d'échanges de lignes.

Exemple :

Après l'appel de la fonction **triangulaire(C)**, on obtient la matrice triangulaire supérieure suivante :

1.	1.	1.	1.	0.	0.	1.	1.	1.	1.
0.	-1.	0.	0.	1.	0.	0.	0.	0.	0.
0.	0.	-1.	-1.	1.	1.	-1.	-1.	0.	0.
0.	0.	0.	-1.	0.	0.	-1.	0.	0.	0.
0.	0.	0.	0.	-1.	0.	-1.	1.	0.	-1.
0.	0.	0.	0.	0.	1.	1.	-1.	0.	0.
0.	0.	0.	0.	0.	0.	2.	-1.	0.	1.
0.	0.	0.	0.	0.	0.	0.	1.	0.	0.
0.	0.	0.	0.	0.	0.	0.	0.	-1.	-1.5
0.	0.	0.	0.	0.	0.	0.	0.	0.	2.

La fonction **triangulaire(C)** renvoie le nombre d'échanges de colonnes dans **C** : **4**

Q 1. d- Écrire la fonction **deteminant (G)**, qui reçoit en paramètre une matrice **G** qui représente une grille binaire carrée. En utilisant la méthode du *pivot de Gauss*, la fonction renvoie la valeur du déterminant de **G**,

Exemple :

La fonction **determinant (G)** renvoie : **-4**

Épreuve d'Informatique – Session 2019 – Filière PSI

Partie IV :

Calcul du nombre de chemins entre deux villes

Dans cette partie, on considère que le module **numpy** est importé :

```
from numpy import *
```

On suppose que le réseau routier est représenté par la matrice symétrique **R**, et que la matrice **R** est créée par la méthode **array()** du module **numpy**.

Exemple :

```
R = array ( [ [0,1,1,1,0,0,0,0], [1,0,1,0,1,0,0,0], [1,1,0,1,1,0,0,0],  
             [1,0,1,0,1,0,1,1], [0,1,1,1,0,1,1,0], [0,0,0,0,1,0,1,1],  
             [0,0,0,1,1,1,0,1], [0,0,0,1,0,1,1,0] ] )
```

Dans cette partie, on propose de résoudre le problème suivant :

Soit **p** un entier strictement positif.

Quel est le nombre de chemins (simples ou non) entre une ville **i** et une ville **j**, passant exactement par **p** routes ?

Par exemple, dans le réseau routier de la **figure 2**, pour aller de la ville **1** à la ville **3**, en passant exactement par **5** routes, on peut trouver plusieurs chemins, dont voici quelques uns :

(1, 4, 5, 7, 6, 3) , **(1, 4, 6, 5, 4, 3)** , **(1, 2, 3, 4, 2, 3)** , **(1, 0, 2, 0, 2, 3)** , ...

Pour résoudre ce problème, on doit procéder ainsi :

Calculer la matrice **M = R^p** (*matrice symétrique **R** élevée à la puissance **p***)

Chaque élément **M[i][j]** représente le nombre de chemins, entre la ville **i** et la ville **j**, passant par **p** routes ;

La matrice **M** est symétrique aussi. Le nombre de chemins, entre la ville **i** et la ville **j**, est le même que celui entre la ville **j** et la ville **i**.

Exemples :

3	1	2	1	3	0	1	1
1	3	2	3	1	1	1	0
2	2	4	2	2	1	2	1
1	3	2	5	2	3	2	1
3	1	2	2	5	1	2	3
0	1	1	3	1	3	2	1
1	1	2	2	2	2	4	2
1	0	1	1	3	1	2	3

$$\mathbf{M} = \mathbf{R}^2$$

4	8	8	10	5	5	5	2
8	4	8	5	10	2	5	5
8	8	8	11	11	5	6	5
10	5	11	8	15	5	11	10
5	10	11	15	8	10	11	5
5	2	5	5	10	4	8	8
5	5	6	11	11	8	8	8
2	5	5	10	5	8	8	4

$$\mathbf{M} = \mathbf{R}^3$$

Par exemple, entre la ville **1** et la ville **7** :

- Il y a **0** chemins qui passent exactement par **deux** routes (*dans la matrice $M = R^2$*) ;
- Il y a **5** chemins qui passent exactement par **trois** routes (*dans la matrice $M = R^3$*).

IV. 12- Produit matriciel

Rappel :

On considère la matrice **A**, de **m** lignes et **n** colonnes, et la matrice **B** de **n** lignes et **p** colonnes. En effectuant le produit matriciel de **A** et **B**, on obtient la matrice **C** de **m** lignes et **p** colonnes, sachant que les coefficients de la matrice **C** sont calculés par la formule suivante :

$$= *$$

Q 12. Écrire la fonction **produit_matriciel(A,B)**, qui reçoit en paramètres deux matrices symétriques **A** et **B**, de même dimension. La fonction calcule et renvoie une nouvelle matrice symétrique **C**, qui contient le produit matriciel de **A** et **B**. *La fonction doit économiser le temps de calcul, en calculant une seule fois les coefficients symétriques dans la matrice **C**.*

IV. 13- Calcul de la puissance par 'exponentiation rapide'

Lorsqu'il s'agit d'une matrice carrée, le calcul de la puissance devient crucial. **L'exponentiation rapide** est un algorithme qui permet de minimiser le nombre de multiplications effectuées dans le calcul de la puissance.

Pour calculer x^n , le principe de l'exponentiation rapide est le suivant :

- Si **n = 1** alors $x^n = x$
- Si **n est pair** alors $x^n = (x^2)^{n/2}$
- Si **n est impair** alors $x^n = x \cdot (x^2)^{(n-1)/2}$

Q 13- Écrire la fonction **puissance(R,n)**, reçoit en paramètres la matrice symétrique **R** représentant un réseau routier, et un entier strictement positif **n**. En utilisant le principe de l'exponentiation rapide, la fonction calcule et renvoie la matrice **Rⁿ**.

Épreuve d'Informatique – Session 2019 – Filière TSI

II- 9. Calcul du nombre de chemins entre deux villes

Dans cette partie, on considère que le module **numpy** est importé :

```
from numpy import *
```

On suppose que le réseau routier est représenté par la matrice symétrique **R**, et que la matrice **R** est créée par la méthode **array()** du module **numpy**.

Exemple :

```
R = array([ [0,1,1,1,0,0,0,0],  
           [1,0,1,0,1,0,0,0],  
           [1,1,0,1,1,0,0,0],  
           [1,0,1,0,1,0,1,1],  
           [0,1,1,1,0,1,1,0],  
           [0,0,0,0,1,0,1,1],  
           [0,0,0,1,1,1,0,1],  
           [0,0,0,1,0,1,1,0] ])
```

Dans cette partie, on propose de résoudre le problème suivant :

Soit **p** un entier strictement positif.

Quel est le nombre de chemins (simples ou non) entre une ville **i** et une ville **j**, passant exactement par **p** routes ?

Exemple :

Dans le réseau routier de la **figure 2**, pour aller de la ville **1** à la ville **3**, en passant exactement par **5** routes, on peut trouver plusieurs chemins, dont voici quelques uns :

- **(1, 4, 5, 7, 6, 3)**
- **(1, 4, 6, 5, 4, 3)**
- **(1, 2, 3, 4, 2, 3)**
- **(1, 0, 2, 0, 2, 3)**
- ...

Pour résoudre ce problème, on doit procéder ainsi :

Calculer la matrice **M = R^p** (*matrice symétrique **R** élevée à la puissance **p***)

Chaque élément **M[i][j]** représente le nombre de chemins, qui partent de la ville **i** à la ville **j**, en passant par **p** routes ;

La matrice **M** est symétrique aussi. Le nombre de chemins, entre la ville **i** et la ville **j**, est le même que celui entre la ville **j** et la ville **i**.

Exemples :

3	1	2	1	3	0	1	1
1	3	2	3	1	1	1	0
2	2	4	2	2	1	2	1
1	3	2	5	2	3	2	1
3	1	2	2	5	1	2	3
0	1	1	3	1	3	2	1
1	1	2	2	2	2	4	2
1	0	1	1	3	1	2	3

$$M = R^2$$

4	8	8	10	5	5	5	2
8	4	8	5	10	2	5	5
8	8	8	11	11	5	6	5
10	5	11	8	15	5	11	10
5	10	11	15	8	10	11	5
5	2	5	5	10	4	8	8
5	5	6	11	11	8	8	8
2	5	5	10	5	8	8	4

$$M = R^3$$

Par exemple, entre la ville **1** et la ville **7** :

- Il y a **0** chemins qui passent exactement par **deux** routes (*dans la matrice $M = R^2$*) ;
- Il y a **5** chemins qui passent exactement par **trois** routes (*dans la matrice $M = R^3$*).

Rappel : Le module *numpy* contient la méthode *dot()* qui calcule le produit matriciel.

Lorsqu'il s'agit d'une matrice carrée, le calcul de la puissance devient crucial. L' **exponentiation rapide** est un algorithme qui permet de minimiser le nombre de multiplications effectuées dans le calcul de la puissance d'une matrice carrée.

Pour calculer x^n , le principe de l'exponentiation rapide est le suivant :

- Si $n = 1$ alors $x^n = x$
- Si n est pair alors $x^n = (x^2)^{n/2}$
- Si n est impair alors $x^n = x \cdot (x^2)^{(n-1)/2}$

Q 9. a- Déterminer la complexité de l'algorithme de l'exponentiation rapide. Justifier votre réponse.

Q 9. b- Écrire la fonction **puissance(R, n)**, reçoit en paramètres la matrice symétrique **R**, qui représente un réseau routier, et un entier **n** strictement positif . La fonction calcule et renvoie la matrice **Rⁿ** (**R** puissance **n**), en utilisant le principe de l'exponentiation rapide.

~~~~~ **FIN DE L'ÉPREUVE** ~~~~~



Les principales méthodes de résolution numérique des **ED** sont séparées en deux grandes catégories :

- **les méthodes à un pas** : Le calcul de la valeur  $y_{n+1}$  au nœud  $t_{n+1}$  fait intervenir la valeur  $y_n$  obtenue à l'abscisse précédente. Les principales méthodes sont celles de : *Euler, Runge-Kutta, Crank-Nicholson ...*
- **les méthodes à multiples pas** : Le calcul de la valeur  $y_{n+1}$  au nœud  $t_{n+1}$  fait intervenir plusieurs valeurs  $y_n, y_{n-1}, y_{n-2}, \dots$  obtenues aux abscisses précédentes. Les principales méthodes sont celles de : *Nyström, Adams-Bashforth, Adams-Moulton, Gear ...*

**Les méthodes d'Adams-Bashforth et d'Adams-Moulton :**

Ce sont des méthodes basées sur des techniques d'intégration numérique qui utilisent les polynômes interpolateurs de Lagrange. La formulation générale de ces méthodes est :

$$y_{n+1} = y_n + h * \sum_{j=-1 \text{ ou } 0}^p b_j * f(t_{n-j}, y_{n-j})$$

Dans la suite de cette partie, nous nous intéressons aux méthodes d'*Adams-Bashforth* et d'*Adams-Moulton* d'ordre 2.

- **Le schéma d'Adams-Bashforth à deux pas, explicite, d'ordre 2 :**

$$\begin{array}{|l} y_0 \text{ donné ;} \\ y_1 \text{ calculé par la méthode Euler ;} \\ y_{n+1} = y_n + \frac{h}{2} (3 * f(t_n, y_n) - f(t_{n-1}, y_{n-1})) \end{array}$$

- **Le schéma d'Adams-Moulton à un pas, implicite, d'ordre 2 :**

$$\begin{array}{|l} y_0 \text{ donné ;} \\ y_{n+1} = y_n + \frac{h}{2} (f(t_n, y_n) + f(t_{n+1}, y_{n+1})) \end{array}$$

En pratique, les schémas d' **Adams-Moulton** ne sont pas utilisés comme tels, car ils sont implicites. On utilise plutôt conjointement un schéma d'**Adams-Moulton** et un schéma d'**Adams-Bashforth** de même ordre, pour construire un schéma *prédicteur-correcteur* : Dans le schéma d' **Adams-Moulton**, on utilise la valeur de  $y_{n+1}$  prédite (calculée) par le schéma d'**Adams-Bashforth**.

**Le schéma prédicteur-correcteur d'ordre 2, d'Adams-Bashforth et Adams-Moulton est :**

$$\begin{array}{|l} y_0 \text{ donné ;} \\ y_1 \text{ calculé par la méthode d'Euler ;} \\ y_{n+1} = y_n + \frac{h}{2} (f(t_n, y_n) + f(t_{n+1}, k)) \\ \text{avec : } k = y_n + \frac{h}{2} (3 * f(t_n, y_n) - f(t_{n-1}, y_{n-1})) \end{array}$$

**Q. 1 :** Écrire une fonction **ABM2** ( $f$ ,  $a$ ,  $b$ ,  $y_0$ ,  $h$ ), qui reçoit en paramètres :

- ☐  $f$  est la fonction qui représente une équation différentielle du premier ordre ;
- ☐  $a$  et  $b$  sont les deux bornes de l'intervalle d'intégration ;
- ☐  $y_0$  est la valeur initiale à l'instant  $a$  ( $y(a) = y_0$ ) ;
- ☐  $h$  est le pas de discrétisation.

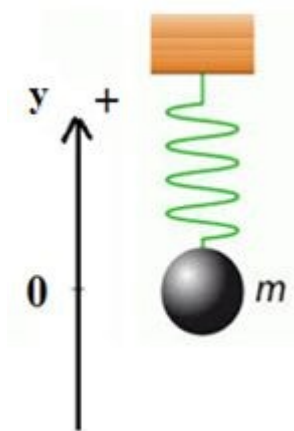
En utilisant le schéma prédicteur-correcteur d'ordre 2, d'Adams-Bashforth et Adams-Moulton, la fonction renvoie **T** et **Y**, qui peuvent être des listes ou des tableaux :

- ☐ **T** contiendra la subdivision de l'intervalle  $[a, b]$ , en valeurs régulièrement espacées, utilisant le pas de discrétisation  $h$  ;
- ☐ **Y** contient les approximations des valeurs  $y(t_i)$ , à chaque instant  $t_i$  de **T**.

### Application à un oscillateur harmonique

Un objet, de masse  $m$ , est suspendu à un ressort fixé à un support.

$0$  représente le point d'équilibre, et  $y(t)$  représente la position de l'objet par rapport au point d'équilibre, avec la direction positive vers le haut.



Si l'objet suspendu est tiré verticalement vers le bas, celui-ci effectue un mouvement harmonique. La forme générale de l'équation différentielle modélisant ce mouvement harmonique est :

$$(E) \quad m * \ddot{y}(t) + b * \dot{y}(t) + k * y(t) = F(t)$$

Les paramètres de cette équation différentielle sont :

- ☐  $b$  : constante de proportionnalité de la force d'amortissement ;
- ☐  $F(t)$  : force extérieure appliquée sur l'objet ;
- ☐  $k$  : constante de rappel du ressort ;
- ☐  $m$  : masse de l'objet.

Si  $b = 0$  et  $F(t) = 0$ , alors le mouvement harmonique est dit : **simple**.

Si  $b \neq 0$  et  $F(t) = 0$ , alors mouvement harmonique est dit : **amorti**.

Si  $F(t) \neq 0$ , alors mouvement harmonique est dit : **forcé**.

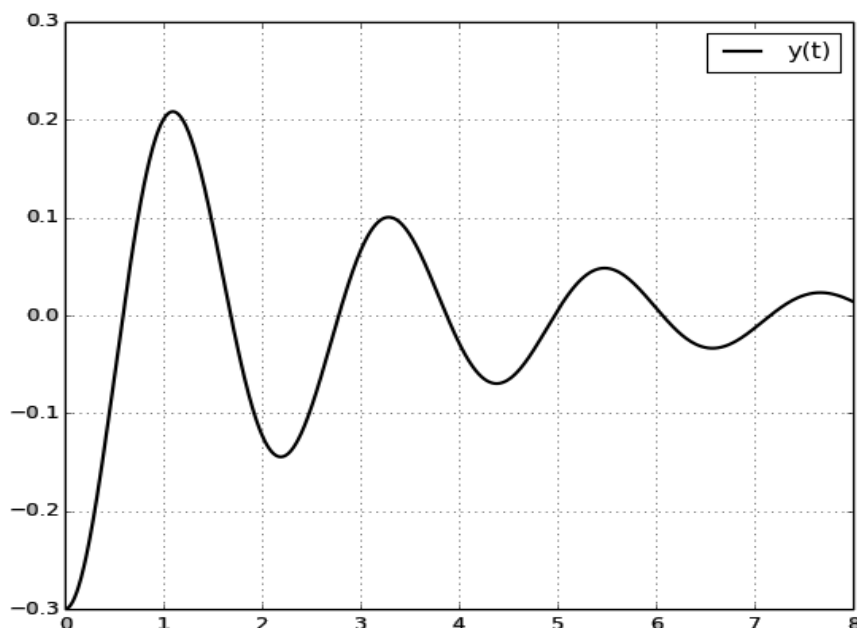
**Q. 2 :** On suppose que les valeurs des paramètres de l'équation différentielle (E), sont :

$$b = 1 \quad ; \quad k = 12.5 \quad ; \quad m = 1.5 \quad \text{et} \quad F(t) = 0$$

Écrire l'équation différentielle (E) sous la forme  $\mathbf{z}' = \mathbf{G}(\mathbf{t}, \mathbf{z})$ , avec  $\mathbf{G}(\mathbf{t}, \mathbf{z})$  une fonction à deux variables  $(\mathbf{t}, \mathbf{z})$  dont  $\mathbf{z}$  est un tableau de longueur 2.

**Q. 3 :** On suppose avoir tiré l'objet suspendu verticalement vers le bas, avant de le relâcher *sans vitesse initiale*.

En utilisant le schéma prédicteur-correcteur d'ordre 2, d'Adams-Bashforth et Adams-Moulton, écrire le code Python qui permet de tracer la courbe ci-dessous, représentant la position  $y(t)$  de l'objet  $m$  suspendu, toutes les  $10^{-3}$  secondes.



**Q. 4 :** Écrire la fonction **racines (P)**, qui reçoit en paramètre la liste (ou le tableau) **P** des positions  $y(t)$  de l'objet suspendu, à intervalles de temps de  $10^3$  secondes. La fonction affiche les valeurs des zéros de la fonction  $y(t)$  : les valeurs des  $t$  tels que  $y(t) = 0$ , ou  $y(t) \approx 0$  avec la précision  $10^{-3}$ .

**Exemple :**

La fonction affiche les valeurs suivantes :

**0.589 , 1.684 , 2.78 , 3.875 , 4.971 , 6.067 , 7.162**

\*\*\*\*\*

## Partie II : Calcul numérique

### ***Résolution numérique d'équation différentielle***

#### **La méthode de Nyström**

Dans cette partie, on suppose que les modules suivants sont importés :

```
from numpy import *
```

```
from matplotlib.pyplot import *
```

Les équations différentielles (**ED**) apparaissent très souvent dans la modélisation de la physique et des sciences de l'ingénieur. Trouver la solution d'une **ED** ou d'un système d'**ED** est ainsi un problème courant, souvent difficile ou impossible à résoudre de façon analytique. Il est alors nécessaire de recourir à des méthodes numériques pour les résoudre.

Le **problème de Cauchy** consiste à trouver une fonction  $y(t)$  définie sur l'intervalle  $[a, b]$  telle que :

$$y' = f(t, y), \quad y(a) = y_0, \quad t \in [a, b];$$

Pour obtenir une approximation numérique de la solution  $y(t)$  sur l'intervalle  $[a, b]$ , nous allons estimer la valeur de cette fonction en un nombre fini de points, pour  $i = 0, 1, \dots, n$ , constituant les nœuds du maillage. La solution numérique obtenue aux points  $t_i$  est notée  $y_i = y(t_i)$ . L'écart entre deux abscisses, noté  $h$ , est appelé : **le pas de discrétisation**.

Les principales méthodes de résolution numérique des **ED** sont séparées en deux grandes catégories :

**les méthodes à un pas** : Le calcul de la valeur  $y_{n+1}$  au nœud  $t_{n+1}$  fait intervenir la valeur  $y_n$  obtenue à l'abscisse précédente. Les principales méthodes sont celles de : *Euler, Runge-Kutta, Crank-Nicholson* ...

**les méthodes à multiples pas** : Le calcul de la valeur  $y_{n+1}$  au nœud  $t_{n+1}$  fait intervenir plusieurs valeurs  $y_n, y_{n-1}, y_{n-2}, \dots$  obtenues aux abscisses précédentes. Les principales méthodes sont celles de : *Nyström, Adams-Bashforth, Adams-Moulton, Gear* ...

#### **La méthode de Nyström :**

La méthode de **Nyström** est une méthode à deux pas, son algorithme est :

$$\begin{cases} y_0 \text{ donné ;} \\ y_1 \text{ calculé par la méthode Euler ;} \\ y_{n+1} = y_n + h \cdot f(t_n, y_n), \end{cases}$$

Géométriquement : on considère la droite de pente  $f(t_n, y_n)$  passant par le point  $(t_n, y_n)$  parallèle à la tangente passant par  $(t_n, y_n)$ . La valeur  $y_{n+1}$  est l'ordonnée du point de cette droite d'abscisse  $t_{n+1}$ .

**Q. 1 :** Écrire une fonction : **nystrom** (**f**, **a**, **b**, **y0**, **h**), qui reçoit en paramètres :

**f** est la fonction qui représente une équation différentielle du premier ordre ;

**a** et **b** sont les deux bornes de l'intervalle d'intégration ;

**y0** est la valeur de la condition initiale à l'instant **a** ( $y(a)=y0$ ) ;

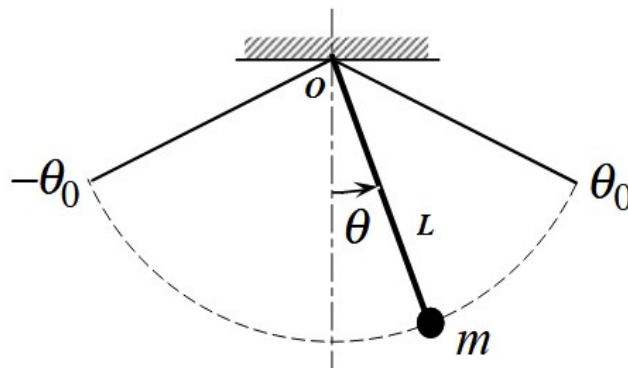
**h** est le pas de discrétisation.

En utilisant le schéma de *Nyström*, la fonction renvoie **T** et **Y**, qui peuvent être des listes ou des tableaux :

**T** contient la subdivision de l'intervalle [**a**, **b**], en valeurs régulièrement espacée, utilisant le **pas** de discrétisation **h** ;

**Y** contient les approximations des valeurs **y(t<sub>i</sub>)**, à chaque instant **t<sub>i</sub>** de **T**.

### **Application au pendule simple**



On considère une masse **m** suspendue par une tige rigide de longueur **L** et de masse négligeable. On désigne par  $\theta$  l'angle entre la verticale passant par le point de suspension et la direction de la tige. On considère que le pendule est également soumis à un frottement fluide de coefficient **k**.

Si l'objet **m** est écarté de façon à ce que l'angle entre la verticale et la tige soit  $\theta_0$ , et ensuite relâché sans vitesse initiale, alors l'objet **m** effectue un mouvement harmonique amorti.

La forme générale de l'équation différentielle modélisant ce mouvement harmonique est :

$$(E) \quad \frac{d^2\theta}{dt^2} + \frac{k}{m} \frac{d\theta}{dt} + \frac{g}{L} \theta = 0$$

Les paramètres de cette équation différentielle sont :

**k** : le coefficient du frottement fluide ;

**g** : la pesanteur ;

**L** : la longueur de la tige ;

**m** : la masse de l'objet suspendu.

**NB :** Cette équation n'a pas de solution analytique, sa solution est numérique.

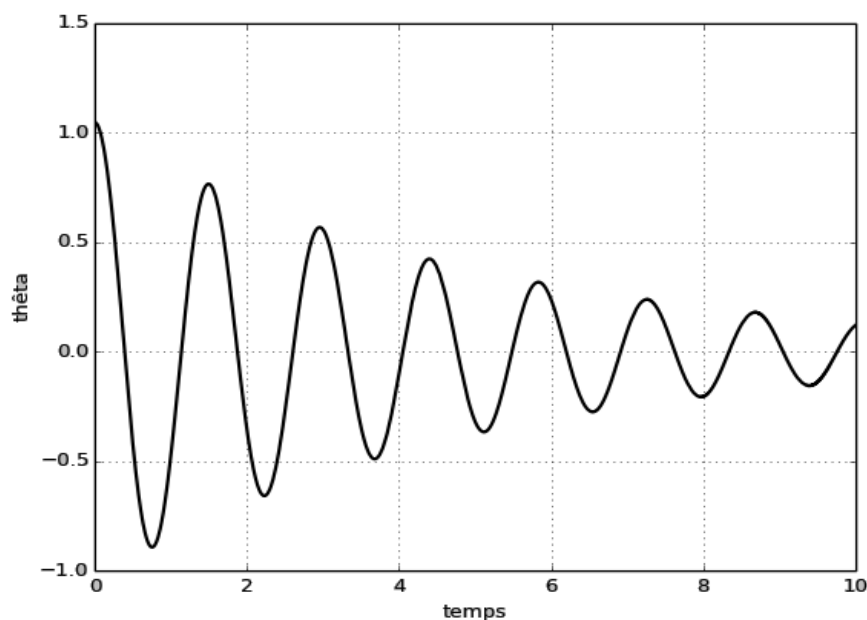
**Q. 2 :** On suppose que les valeurs des paramètres de l'équation différentielle (E), sont :

$$m = 1 \text{ kg} ; \quad g = 9.81 \text{ m/s}^2 ; \quad L = 0.50 \text{ m} \quad \text{et} \quad k = 0.1 \text{ kg/s}$$

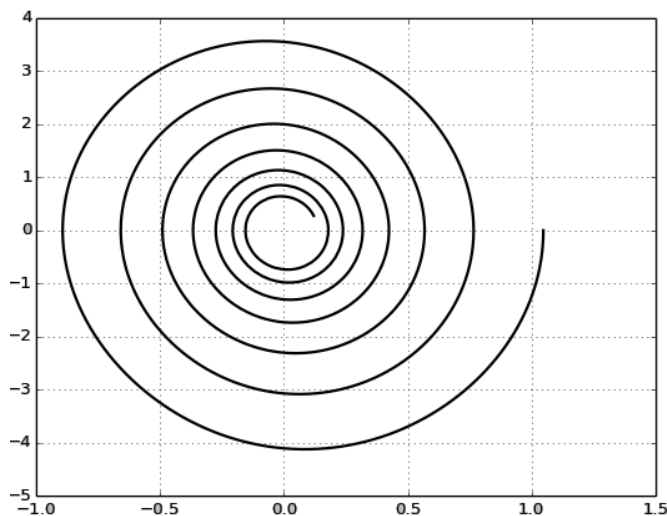
Écrire l'équation différentielle (E) sous la forme  $\mathbf{z}' = \mathbf{F}(\mathbf{z}, t)$ , avec  $\mathbf{F}(\mathbf{z}, t)$  une fonction à deux variables  $(\mathbf{z}, t)$  dont  $\mathbf{z}$  est un tableau de longueur 2.

**Q. 3 :** On suppose avoir écarté l'objet, de façon à ce que l'angle entre la verticale et la tige, soit de  $\pi/3$ , avant de le relâcher *sans vitesse initiale*.

**3.a-** En utilisant la fonction de Nyström, écrire le code python qui, permet de tracer le graphe ci-dessous, qui représente la valeur de l'angle  $\theta$  toutes les  $10^{-3}$  secondes.



**3.b-** Écrire le code python permettant de tracer la courbe ci-dessous, qui représente le **portrait de phase du pendule** : les valeurs de  $\dot{\theta}$  (axe des ordonnées) en fonction des valeurs de  $\theta$  (axe des abscisses).

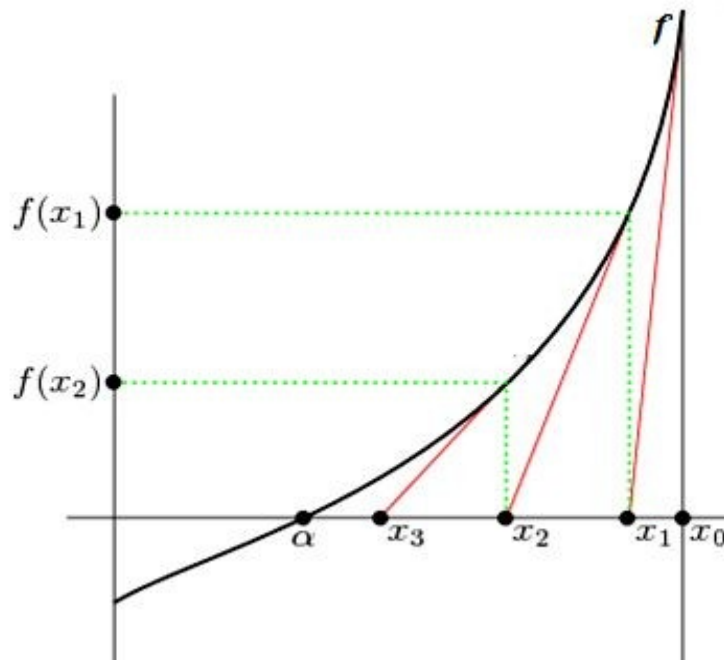


~ ~ ~ ~ ~

**Partie III : Calcul numérique**

***Calcul de la racine  $n^{ième}$  par la méthode de 'Newton'***

De nombreux problèmes d'économie, de mathématiques ou de physique se concluent par la résolution d'une équation  $f(x) = 0$ . Bien souvent, il n'est pas possible de résoudre exactement cette équation, et on cherche une valeur approchée de la solution (ou des solutions). **Newton** a proposé une méthode générale pour obtenir une telle approximation. L'idée est de remplacer la courbe représentative de la fonction par sa tangente.



On part d'un point  $x_0$  de l'intervalle de définition de  $f$ , et on considère la tangente à la courbe représentative de  $f$  en  $(x_0, f(x_0))$ . On suppose que cette tangente coupe l'axe des abscisses (c.à.d.  $f'(x_0)$  non nul).

Soit  $x_1$  l'abscisse de l'intersection de la tangente avec l'axe des abscisses : 
$$x_1 = x_0 - \frac{f(x_0)}{f'(x_0)}$$

Puisque la tangente est proche de la courbe, on peut espérer que  $x_1$  donne une meilleure estimation d'une solution de l'équation  $f(x) = 0$  que  $x_0$ .

On recommence alors le procédé à partir de  $x_1$ , on obtient  $x_2$  : 
$$x_2 = x_1 - \frac{f(x_1)}{f'(x_1)}$$

...

Ainsi, on construit par récurrence une suite  $(x_n)$  définie par : 
$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

Sous de bonnes hypothèses sur  $f$ , assez restrictives,  $(x_n)$  converge vers la solution de l'équation  $f(x) = 0$ , et la convergence est très rapide.

Approximation de la dérivée par la moyenne des taux d'accroissement à gauche et à droite :

Si  $f$  est une fonction dérivable en  $x_i$ , on peut toutefois obtenir une approximation de  $f'(x_i)$  par la moyenne des taux d'accroissement à gauche et à droite de  $x_i$  :

Pour un réel  $h > 0$  (très proche de 0) :

$$f'(x_i) \approx \frac{f(x_i + h) - f(x_i - h)}{2h}$$

**Q.1-** Montrer que :  $x_{n+1} = x_n - 2h \frac{f(x_n)}{f(x_n + h) - f(x_n - h)}$

**Q.2-** Écrire une fonction : **newton** ( $f$ ,  $x_0$ , **eps**,  $h$ ) qui reçoit en paramètres :

- ☐ La fonction  $f$  ;
- ☐ Le premier terme  $x_0$  de la suite  $(x_n)$  ;
- ☐ Le réel  $h$  strictement positif très proche de 0 ;
- ☐ Le réel **eps** strictement positif qui représente la précision.

La fonction renvoie le premier terme  $x_k$  de la suite  $(x_n)$  tel que :  $|x_k - x_{k-1}| \leq \mathbf{eps}$

Application : Calcul de la racine n-ème positive d'un réel positif

Pour calculer la racine n-ème positive d'un réel positif  $a$ , il suffit de trouver la solution positive de l'équation :  $x^n - a = 0$ , par la méthode de Newton.

Dans ce qui suit, on suppose que  $a$  et  $n$  sont deux variables globales déclarées et initialisées :

$a$  est un réel positif, et  $n$  un entier strictement positif.

**Q. 3-** Écrire la fonction **g** ( $x$ ), qui reçoit en paramètre un réel positif  $x$  et qui renvoie :  $x^n - a$ .

**Q. 4-** On suppose que les modules suivant sont importé :

**from numpy import \***

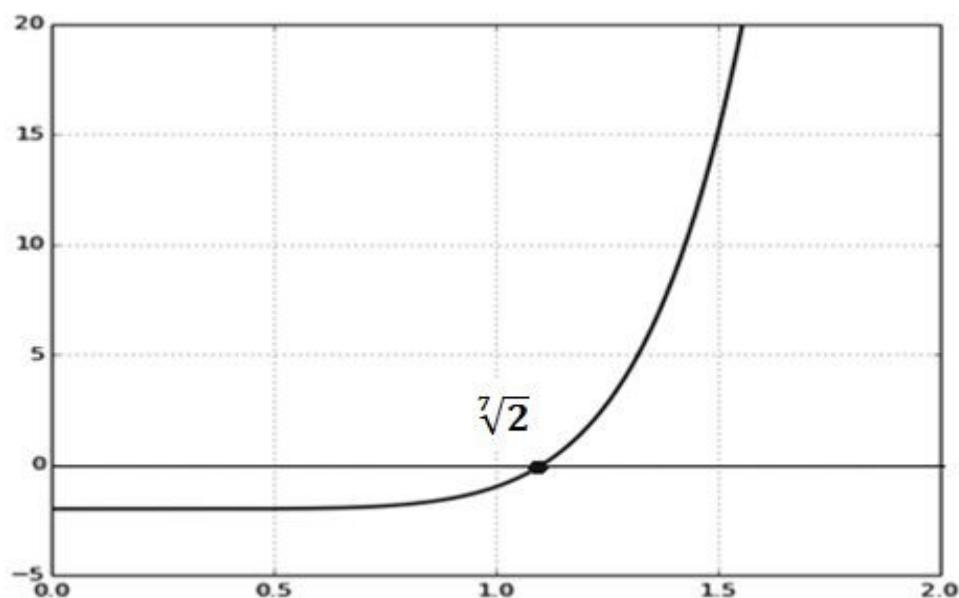
**from matplotlib.pyplot import \***

Écrire le code du programme Python qui permet de tracer la courbe de la fonction **g**, dans l'intervalle  $[0, a]$ .

Le nombre de points générés dans la courbe est : **500**

**Exemple :**

Pour **a=2.0** et **n=7**, on obtient la courbe suivante :



Représentation graphique de la fonction :  $g(x) = x^7 - 2$

**Q. 5-** Écrire le code du programme Python qui utilise la fonction **newton()**, et qui affiche la valeur approximative de la racine n-ème positive de  $a$  :  $\sqrt[n]{a}$ , avec la précision  $10^{-12}$  et  $h = 10^{-5}$ .

**Exemple :**

Pour  $n=7$  et  $a=2.0$ , le programme affiche la racine 7<sup>ème</sup> positive de 2 : **1.1040895136738123**

~ ~ ~ ~ ~ **FIN** ~ ~ ~ ~ ~

## **Partie III : Calcul scientifique**

### **Calcul de la matrice inverse d'une matrice carrée inversible**

#### *Méthode : Élimination de 'Gauss-Jordan'*

En mathématiques, l'élimination de « Gauss-Jordan », aussi appelée pivot de Gauss, nommée en hommage à **Carl Friedrich Gauss** et **Wilhelm Jordan**, est un algorithme de l'algèbre linéaire pour déterminer les solutions d'un système d'équations linéaires, pour calculer l'inverse d'une matrice carrée inversible ou pour déterminer le rang d'une matrice...

Pour calculer la matrice inverse d'une matrice carrée  $M$  inversible, la méthode de l'élimination de « Gauss-Jordan » consiste à effectuer des transformations, simultanément sur les deux matrices : la matrice  $M$ , et la matrice identité  $E$  de même dimension que  $M$ .

*Le but de la méthode de l'élimination de « Gauss-Jordan » est de transformer la matrice  $M$  en matrice identité, tout en effectuant simultanément les mêmes opérations (effectuer les mêmes échanges, utiliser les mêmes valeurs des pivots ...), sur les deux matrices  $M$  et  $E$ . La matrice  $E$  contient, alors, la matrice inverse de la matrice  $M$  initiale.*

#### **Exemple :**

Matrice carrée inversible  $M$  :

$$\begin{bmatrix} 1. & -1. & 2. & -2. \\ 3. & 2. & 1. & -1. \\ 2. & -3. & -2. & 1. \\ -1. & -2. & 3. & -3. \end{bmatrix}$$

Matrice identité  $E$ , de même dimension que  $M$  :

$$\begin{bmatrix} 1. & 0. & 0. & 0. \\ 0. & 1. & 0. & 0. \\ 0. & 0. & 1. & 0. \\ 0. & 0. & 0. & 1. \end{bmatrix}$$

Après les transformations, les deux matrices  $M$  et  $E$  deviennent ainsi :

Matrice  $M$  transformée en identité :

$$\begin{bmatrix} 1. & 0. & 0. & 0. \\ 0. & 1. & 0. & 0. \\ 0. & 0. & 1. & 0. \\ 0. & 0. & 0. & 1. \end{bmatrix}$$

$E$  contient l'inverse de la matrice  $M$  initiale :

$$\begin{bmatrix} 0.8 & -0.1 & 0. & -0.5 \\ -1. & 0.5 & 0. & 0.5 \\ 5. & -2. & -1. & -3. \\ 5.4 & -2.3 & -1. & -3.5 \end{bmatrix}$$

On suppose que le module **numpy** est importé :

```
import numpy as np
```

Le module **numpy** contient les méthodes et les fonctions permettant de manipuler les matrices.

*On rappelle que les indices des listes et tableaux en Python commencent à **0**.*

#### **Exemple :**

```
M = np.array( [ [ 1, -1, 2, -2 ], [ 3, 2, 1, -1 ], [ 2, -3, -2, 1 ], [ -1, -2, 3, -3 ] ], float )
```

Cette instruction permet de créer la matrice **M** suivante :

$$\begin{bmatrix} 1. & -1. & 2. & -2. \\ 3. & 2. & 1. & -1. \\ 2. & -3. & -2. & 1. \\ -1. & -2. & 3. & -3. \end{bmatrix}$$

**III. 1- Matrice inversible :**

Une matrice carrée est inversible, si son déterminant est différent de 0.

Le module **numpy** contient un sous-module appelé : **linalg**. Ce dernier contient la méthode : **det ()** qui reçoit en paramètre une matrice, et qui retourne la valeur du déterminant de cette matrice.

Écrire la fonction : **inversible (M)**, qui reçoit en paramètre une matrice carrée **M**, et qui retourne **True** si la matrice M est inversible, sinon, elle retourne **False**.

**III. 2- Matrice identité :**

Écrire la fonction : **identite (n)**, qui reçoit en paramètre un entier strictement positif n, et qui crée et retourne la matrice identité d'ordre n (n lignes et n colonnes), de nombres réels.

Exemple :

La fonction **identite (4)** retourne la matrice identité E d'ordre 4 suivante :

$$\begin{bmatrix} 1. & 0. & 0. & 0. \\ 0. & 1. & 0. & 0. \\ 0. & 0. & 1. & 0. \\ 0. & 0. & 0. & 1. \end{bmatrix}$$

**III. 3- Opération de transvection :**

Écrire une fonction : **transvection (M, p, q, r)**, qui reçoit en paramètres une matrice **M**, deux entiers **p** et **q** représentant les indices de deux lignes dans la matrice **M**, et un nombre réel **r**. Dans la matrice M, la fonction remplace la ligne **p** par la ligne résultat de la combinaison linéaire des deux lignes **p** et **q**, suivante : **Mp+r\*Mq**.

Exemple :

Matrice M :

$$\begin{bmatrix} 1. & -1. & 2. & -2. \\ 3. & 2. & 1. & -1. \\ 2. & -3. & -2. & 1. \\ -1. & -2. & 3. & -3. \end{bmatrix}$$

Après l'appel : **transvection (M, 2, 0, -3.)**, la matrice M devient :

$$\begin{bmatrix} 1. & -1. & 2. & -2. \\ 3. & 2. & 1. & -1. \\ -1. & 0. & -8. & 7. \\ -1. & -2. & 3. & -3. \end{bmatrix}$$

**III. 4- Échanger deux lignes dans une matrice :**

Écrire la fonction : **echange\_lignes (M, p, q)**, qui reçoit en paramètres une matrice **M**, et deux entiers **p** et **q** représentant respectivement deux indices de deux lignes dans la matrice **M**. la fonction échange les lignes **p** et **q** dans la matrice **M**.

Exemple :

Matrice M :

$$\begin{bmatrix} 1. & -1. & 2. & -2. \\ 3. & 2. & 1. & -1. \\ 2. & -3. & -2. & 1. \\ -1. & -2. & 3. & -3. \end{bmatrix}$$

Après l'appel : **echanger\_lignes (M, 1, 3)**, les lignes 1 et 3 de la matrice M sont échangées :

$$\begin{bmatrix} 2. & -3. & -2. & 1. \\ 1. & -1. & 2. & -2. \\ 3. & 2. & 1. & -1. \\ -1. & -2. & 3. & -3. \end{bmatrix}$$

**III. 5- Recherche de la ligne du pivot dans une colonne de la matrice :**

Écrire la fonction : **ligne\_pivot** (**M**, **c**), qui reçoit en paramètres une matrice **M**, et un entier **c** représentant une colonne dans la matrice **M**.

La fonction retourne l'indice **p** tel que :  $|M[p, c]| = \max \{ |M[i, c]| \mid 0 \leq i \leq c \}$

**Exemple :**

Si la matrice M est la suivante :

$$\begin{bmatrix} -1. & 1. & 5. & 2. \\ 10. & 1. & -6. & -4. \\ 8. & 6. & -3. & 7. \\ -9. & 5. & 7. & -2. \end{bmatrix}$$

La fonction **ligne\_pivot** (**M**, **2**) retourne l'indice : **1**, car, dans la colonne 2, l'élément -6.0 est le plus grand en valeur absolue, parmi les éléments des lignes 0, 1 et 2.

**III. 6- Transformer la matrice M en matrice triangulaire inférieure :**

Écrire la fonction : **triangulaire\_inf** (**M**, **E**), qui reçoit en paramètre une matrice carrée inversible **M**, et une matrice identité **E** de même dimension que **M**. La fonction transforme la matrice **M** en matrice triangulaire inférieure. Les transformations effectuées sur la matrice **M**, doivent être effectuées simultanément sur la matrice **E**.

**Exemple :**

Après l'appel de la fonction **triangulaire\_inf** (**M**, **E**), les matrices M et E deviennent ainsi :

Matrice M triangulaire inférieure

$$\begin{bmatrix} 1.25 & 0. & 0. & 0. \\ 3.33 & 2.67 & 0. & 0. \\ 1.67 & -3.67 & -1. & 0. \\ -1. & -2. & 3. & -3. \end{bmatrix}$$

Matrice E

$$\begin{bmatrix} 1. & -0.12 & 0. & -0.62 \\ 0. & 1. & 0. & -0.33 \\ 0. & 0. & 1. & 0.33 \\ 0. & 0. & 0. & 1. \end{bmatrix}$$

**III. 7- Transformer la matrice triangulaire inférieure M en matrice diagonale :**

Les deux matrices **M** et **E** sont les résultats de la fonction **triangulaire\_inf** (**M**, **E**). La matrice **M** est devenue triangulaire inférieure. Écrire la fonction : **diagonale** (**M**, **E**), qui transforme la matrice **M** en matrice diagonale. Les transformations effectuées sur la matrice **M**, doivent être aussi effectuées simultanément sur la matrice **E**.

**Exemple :**

Après l'appel de la fonction **diagonale** (**M**, **E**), les matrices M et E deviennent ainsi :

Matrice M diagonale

$$\begin{bmatrix} 1.25 & 0. & 0. & 0. \\ 0. & 2.67 & 0. & 0. \\ 0. & 0. & -1. & 0. \\ 0. & 0. & 0. & -3. \end{bmatrix}$$

Matrice E

$$\begin{bmatrix} 1. & -0.12 & 0. & -0.62 \\ -2.67 & 1.33 & 0. & 1.33 \\ -5. & 2. & 1. & 3. \\ -16.2 & 6.9 & 3. & 10.5 \end{bmatrix}$$

**III. 8- Calcul de la matrice inverse par élimination de « Gauss-Jordan » :**

Écrire la fonction : **inverse(M)**, qui reçoit en paramètre une matrice carrée inversible **M**, et qui calcule et renvoie la matrice inverse de **M**. (Ne pas utiliser la méthode prédéfinie `inv()` de Python)

**Exemple :**

M matrice carrée inversible :

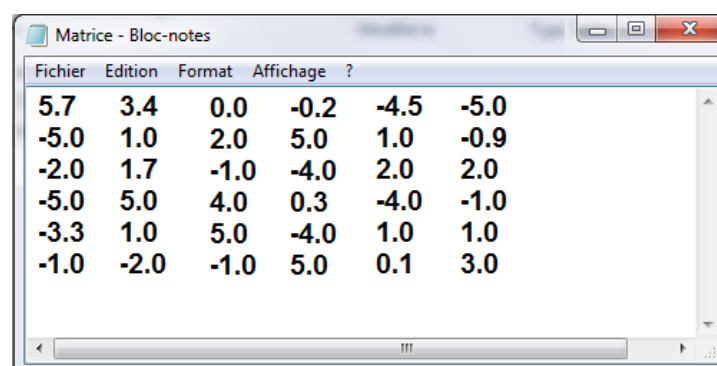
$$\begin{bmatrix} 1. & -1. & 2. & -2. \\ 3. & 2. & 1. & -1. \\ 2. & -3. & -2. & 1. \\ -1. & -2. & 3. & -3. \end{bmatrix}$$

La fonction **inverse(M)** renvoie la matrice inverse de M :

$$\begin{bmatrix} 0.8 & -0.1 & 0. & -0.5 \\ -1. & 0.5 & 0. & 0.5 \\ 5. & -2. & -1. & -3. \\ 5.4 & -2.3 & -1. & -3.5 \end{bmatrix}$$

**III. 9- Matrice stockée dans un fichier texte :**

On suppose avoir créé un fichier texte contenant une matrice carrée. Chaque ligne du fichier contient une ligne de la matrice, et les éléments de chaque ligne du fichier sont séparés par le caractère espace.

**Exemple :**

| Fichier | Edition | Format | Affichage | ?    |      |
|---------|---------|--------|-----------|------|------|
| 5.7     | 3.4     | 0.0    | -0.2      | -4.5 | -5.0 |
| -5.0    | 1.0     | 2.0    | 5.0       | 1.0  | -0.9 |
| -2.0    | 1.7     | -1.0   | -4.0      | 2.0  | 2.0  |
| -5.0    | 5.0     | 4.0    | 0.3       | -4.0 | -1.0 |
| -3.3    | 1.0     | 5.0    | -4.0      | 1.0  | 1.0  |
| -1.0    | -2.0    | -1.0   | 5.0       | 0.1  | 3.0  |

Matrice carrée inversible d'ordre 6, stockée dans un fichier texte

Écrire la fonction : **matrice\_fichier(ch)**, qui reçoit en paramètre une chaîne de caractère **ch**, qui contient le chemin absolu du fichier texte, contenant une matrice carrée :

Si cette matrice est inversible, la fonction renvoie sa matrice inverse ;

Si cette matrice n'est pas inversible, la fonction affiche le message « Matrice non inversible » ;

Si le fichier texte ne se trouve pas à l'emplacement spécifié dans la chaîne **ch**, l'exception **FileNotFoundError** sera levée. La fonction affiche le message : « Fichier texte inexistant »

----- FIN DE L'ÉPREUVE -----

## Epreuve CNC - Session 2016 - Filière MP / PSI / TSI

L'énoncé de cette épreuve, commune aux candidats des filières MP/ PSI/ TSI,  
comporte 10 pages.

L'usage de la calculatrice est **interdit**.

Les candidats sont informés que la précision des raisonnements **algorithmiques** ainsi que le soin apporté à la rédaction et à la présentation des copies seront des éléments pris en compte dans la notation. Il convient en particulier de rappeler avec précision les **références** des questions abordées. Si, au cours de l'épreuve, un candidat repère ce qui peut lui sembler être une erreur d'énoncé, il le signale sur sa copie et poursuit sa composition en expliquant les raisons des initiatives qu'il est amené à prendre.

### Remarques générales :

- L'épreuve se compose de deux problèmes indépendants.
- Toutes les instructions et les fonctions demandées seront écrites en **Python**.
- Les questions non traitées peuvent être admises pour aborder les questions ultérieures.
- Toute fonction peut être décomposée, si nécessaire, en plusieurs fonctions

## PROBLÈME I : CALCUL SCIENTIFIQUE



### Méthodes à un pas :

Nous allons nous intéresser dans ce problème, à quelques méthodes permettant de résoudre numériquement les équations différentielles **du premier ordre**, avec une condition initiale, sous la forme :

$$\begin{cases} y'(t) = f(t, y(t)) \\ y(t_0) = y_0 \end{cases}$$

On note  $I = ]t_0, t_0 + T[$  l'intervalle de résolution. Pour un nombre de noeuds  $N$  donné, soit  $t_n = t_0 + nh$ , avec  $n = 0, 1, 2, \dots, N$ , une suite de noeuds de  $I$  induisant une discrétisation de  $I$  en sous-intervalles  $I_n = [t_n, t_{n+1}]$ . La longueur  $h$  de ces sous-intervalles est appelée pas de discrétisation, le pas  $h$  de discrétisation est donné par  $h = \frac{T}{N}$ . Soit  $y_j$  l'approximation au noeud  $t_j$  de la solution exacte  $y(t_j)$ . Les méthodes de résolution numériques, étudiées dans ce problème, s'écrivent sous la forme :

$$\begin{cases} y_{n+1} = y_n + h\Phi(t_n, y_n, h) \\ t_{n+1} = t_n + h \end{cases}$$

Avec

$$\Phi(t, y, h) = \alpha f(t, y) + \beta f(t + h, y + hf(t, y));$$

où  $\alpha, \beta$  sont des constantes comprises entre 0 et 1.

### Question 1 :

Pour quelles valeurs du couple  $(\alpha, \beta)$  retrouve-t-on la méthode d'Euler ?

Dans la suite on considère une deuxième méthode dite de **Heun**, cette méthode correspond aux valeurs du couple  $(\alpha, \beta) = (\frac{1}{2}, \frac{1}{2})$ .

### Question 2 :

Écrire une fonction de prototype **def Heun(f, t<sub>0</sub>, y<sub>0</sub>, T, N)** : qui prend en paramètres la fonction  $f : (t, y) \rightarrow f(t, y)$ , la condition initiale  $t_0, y_0$  la valeur de  $f$  en  $t_0$  le nombre de nœuds  $N$  et la valeur final du temps  $T$  et qui retourne deux listes  $tt = [t_0, t_1, \dots, t_N]$  et  $y_h = [y_0, y_1, \dots, y_N]$  donné par le schéma :

$$\begin{cases} y_{n+1} = y_n + h\Phi(t_n, y_n, h) \\ t_{n+1} = t_n + h \end{cases}$$

Avec

$$\Phi(t, y, h) = \frac{1}{2}f(t, y) + \frac{1}{2}f(t + h, y + hf(t, y));$$

Le pas de discrétisation  $h$  est donné par  $h = \frac{T}{N}$ .

**Remarque :** La valeur retournée par la fonction **Heun** pourra être un couple constitué de deux listes, un tableau constitué de deux listes (par exemple un type *array* du module *numpy*) ou tout autre structure de données constitué de deux listes.

### Application au circuit RLC :

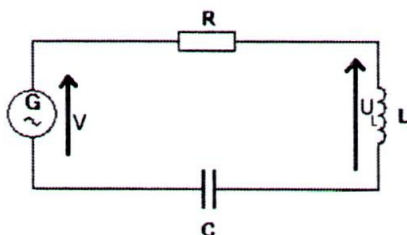


FIGURE 1: circuit RLC

On souhaite travailler sur un circuit *RLC* en série qui sera constitué des éléments suivants : une résistance  $R(I\Omega)$ , une inductance  $L(1H)$ , une capacité  $C(1F)$  et un générateur qui délivrera une source de tension sinusoïdale  $V(t) = \cos(2\pi t)$ .

On s'intéresse à la tension  $U_L$  aux bornes de la bobine qui satisfait l'équation différentielle :

$$\ddot{U}_L + \dot{U}_L + U_L = -4\pi^2 \cos(2\pi t) \quad (I)$$

**Question 3 :**

Écrire une fonction de prototype **def Euler(f, t<sub>0</sub>, T, y<sub>0</sub>, N)** : qui prend en paramètres la fonction  $f : (t, y) \rightarrow f(t, y)$ , la condition initiale  $t_0$ , la valeur de  $f$  en  $t_0$ , le nombre de nœuds  $N$  et la valeur final du temps  $T$  et qui retourne deux listes  $tt = [t_0, t_1, \dots, t_N]$  et  $y_h = [y_0, y_1, \dots, y_N]$  donné par le schéma d'*Euler* permettant de résoudre des équations différentielles du premier ordre.

**Question 4 :**

Donner la complexité de la fonction d'*Euler* en expliquant le descriptif du calcul.

On considère les valeurs suivantes :  $y(t) = U_I(t)$  et  $z(t) = \dot{U}_I(t)$ . L'équation différentielle (I) est du second ordre.

**Question 5 :**

Définissez un système de deux équations différentielles du premier ordre qui puisse satisfaire l'équation (I).

**Question 6 :**

En posant  $Y(t) = [U_I(t), \dot{U}_I(t)]$ , montrez que l'équation (I) peut s'écrire sous la forme :

$$Y'(t) = F(t, Y(t)) \quad (\text{II})$$

avec  $F : t, X \rightarrow F(t, X)$  est une fonction à préciser ( $X = [X[0], X[1]]$ ).

**Question 7 :**

Implémenter en Python l'équation différentielle (II) en utilisant la fonction *odeint()* du module *scipy.integrate*, la méthode d'*Euler* et celle de *Heun*. On affichera dans une même fenêtre le résultat de ses trois méthodes avec les valeurs suivantes :

$N = 1000$ ,  $t_0 = 0$ ,  $T = 3$ ,  $U_I(0) = 0$  et  $\dot{U}_I(0) = 0$ . Voir les résultats sur la *figure 1* se trouvant dans l'annexe du document.

On ajoutera quatre types d'informations :

- le titre de la figure : 'circuit RLC, tension bobine'
- un label associé à l'axe des abscisses : 'temps (s)'
- un label associé à l'axe des ordonnées : 'tension (mV)'
- une légende associée au graphique : '*Odeint*' associé à la méthode d'*odeint*
- une légende associée au graphique : '*Euler*' associé à la méthode d'*Euler*
- une légende associée au graphique : '*Heun*' associé à la méthode d'*Heun*

## Annexe

### A) Image du document

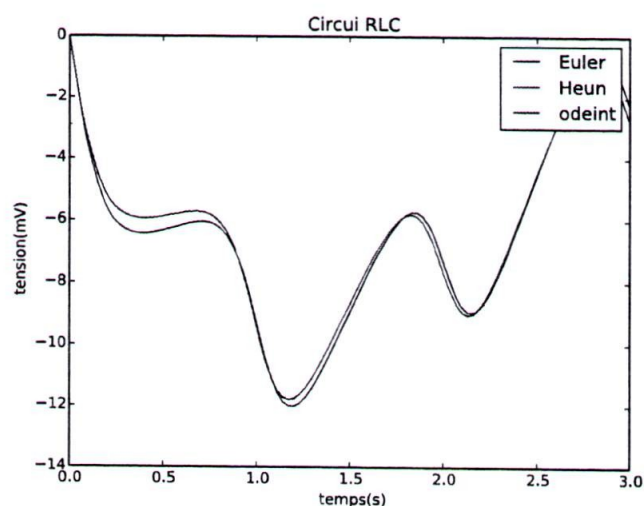


FIGURE 1: circuit RLC

### B) Syntaxe de quelques fonctions du langage Python

#### 1) `matplotlib.pyplot.xlabel`

`matplotlib.pyplot.xlabel(s, *args, **kwargs)`

**Parameters :** `s` : string.

**Returns :** Set the x axis label of the current axi.

#### 2) `matplotlib.pyplot.ylabel`

`matplotlib.pyplot.ylabel(s, *args, **kwargs)`

**Parameters :** `s` : string.

**Returns :** Set the y axis label of the current axi.

### 3) **matplotlib.pyplot.plot**

**matplotlib.pyplot.plo**(\*args, \*\*kwargs)

**Parameters :** **\*args :**

args is a variable length argument, allowing for multiple x, y pairs with an optional format string.

**y0 :** array

Initial condition on y (can be a vector).

**character : description**

'-' : solid line style

'--' : dashed line style

'-.' : dash-dot line style

'.' : dotted line style

'.' : point marker

'.' : pixel marker

'o' : circle marker

'v' : triangle\_down marker

**character : color**

'b' : blue

'g' : green

'r' : red

**Returns** Return value is a list of lines that were added.

### 4) **scipy.integrate.odeint**

**scipy.integrate.odeint**(func, y0, t, args=(), Dfun=None, col\_deriv=0, full\_output=0, ml=None, mu=None, rtol=None, atol=None, tcrit=None, h0=0.0, hmax=0.0, hmin=0, mxstep=0, mxhnil=0, mxordn=12, mxords=5, printmessg=0)

### 5) **matplotlib.pyplot.legend**

**matplotlib.pyplot.legend**(\*args, \*\*kwargs)

**Parameters :**

**Returns :** Places a legend on the axes.

To make a legend for lines which already exist on the axes (via plot for instance), simply call this function with an iterable of strings, one for each legend item.

### 6) **matplotlib.pyplot.title**

**matplotlib.pyplot.title**(s, \*args, \*\*kwargs)



FIN DE L'ÉPREUVE

**Exercice 2. Thème : calcul scientifique**

On souhaite résoudre l'équation différentielle :

$$\frac{d^2 y}{dt^2} = ay + b \frac{dy}{dt} \quad (E)$$

avec :  $a = -3.0$  et  $b = -1.0$

**Question 6 :**

La fonction **`odeint()`** du module **`scipy.integrate`** a pour syntaxe :

**`scipy.integrate.odeint (fonction f, valeur initiale en a, subdivision de [a, b])`**.

La fonction **`odeint()`** ne permet de résoudre que des équations d'ordre 1.

Expliquer comment on peut transformer l'équation (E) en une équation du premier ordre.

**Question 7 :**

Le principe d'utilisation de la fonction **`odeint()`** permet d'obtenir une estimation numérique de la

solution du problème de Cauchy : 
$$\begin{cases} Y'(t) = F(Y(t), t) \\ Y(t_0) = Y_0 \end{cases}$$

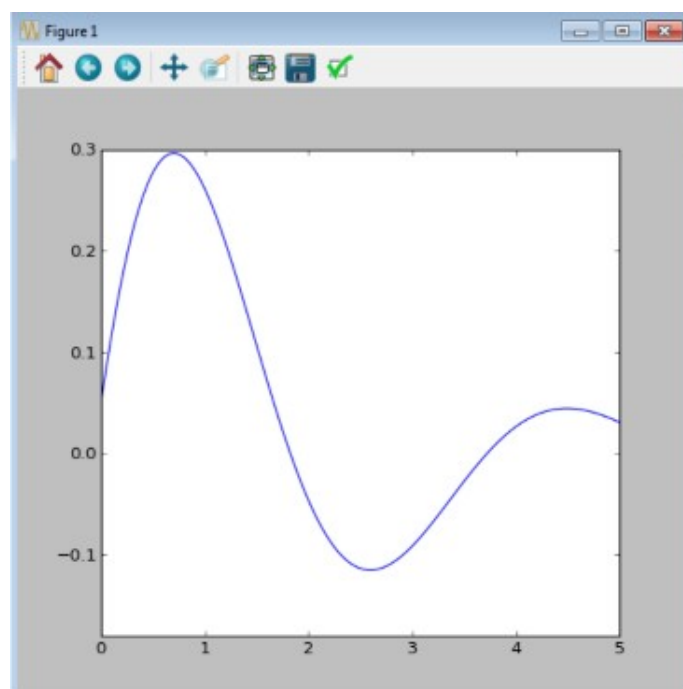
Écrire en **Python** une fonction **`F(z,t)`** qui prend en argument un vecteur colonne **`z`** composé de deux éléments, la variable **`t`** et qui retourne un tableau (array du module **`numpy`**) composé également de deux éléments.

La librairie **`numpy`** est très riche en fonctionnalités (**`np.arange()`**, **`np.linspace()`**, **`np.array()`**, **`np.ones()`**, **`np.sin()`**, **`np.histogram()`**, **`np.concatenate()`**, ...)

Lors de l'implémentation de la fonction **`F()`**, préciser bien à quoi correspondent les différents paramètres mis en œuvre.

**Question 8 :**

On souhaite avoir la représentation suivante de cette équation différentielle :



Pour la courbe représentée dans la "Figure 1" le nombre d'échantillons générés est égal à 100.

Implémenter dans le langage **Python**, en utilisant la fonction **odeint()** citée précédemment, la formule (E) générant le graphique de la "Figure 1".

Attention vous devez pour les valeurs initiales qui vous manquent estimer leurs valeurs en observant la courbe de cette figure.

La librairie **matplotlib** permet de tracer toutes sortes de graphiques notamment avec la sous-librairie **matplotlib.pyplot**.

## ANNEXE

**str(p)** : Retourne une chaîne contenant une représentation bien imprimable d'un objet **p**. Pour les chaînes, cela renvoie la chaîne elle-même.

### Opérations sur les listes :

Soit **L** un élément de type **list**. La liste des méthodes des objets de type **list** est :

**list.append(x)** Ajoute un élément à la fin de la liste.

**list.extend(L)** Étend la liste en y ajoutant tous les éléments de la liste fournie.

**list.insert(i, x)** Insère un élément à la position indiquée. Le premier argument est la position de l'élément courant avant lequel l'insertion doit s'effectuer.

**list.remove(x)** Supprime de la liste le premier élément dont la valeur est x. Une exception est levée s'il n'existe aucun élément avec cette valeur.

**list.index(x)** Retourne la position du premier élément de la liste ayant la valeur x. Une exception est levée s'il n'existe aucun élément avec cette valeur.

**list.count(x)** Retourne le nombre d'éléments ayant la valeur x dans la liste.

**list.sort(cmp=None, key=None, reverse=False)** Trie les éléments de la liste.

**list.reverse()** Inverse l'ordre des éléments de la liste en place.

Figure 1 exercice 2

